
UNIT 12 IMPLEMENTATION STRATEGIES-1

Structure

Page no.

- 12.0 Introduction
- 12.1 Objectives
- 12.2 Mapping Design to Code
- 12.3 Creating Class Definition from Class Diagram
- 12.4 Implementing Associations
- 12.5 Unidirectional Implementations
 - 12.5.1 Optional Associations
 - 12.5.2 One-to-One Associations
 - 12.5.3 Associations with Multiplicity 'Many'
- 12.6 Bi-directional Implementations
 - 12.6.1 One-to-One and Optional Associations
 - 12.6.2 One-to-Many Associations
 - 12.6.3 Immutable Association
- 12.7 Summary
- 12.8 Solutions/Answer to Check Your Progress
- 12.9 References/Further Reading

12.0 INTRODUCTION

You are aware of the object design models. Object design comes between the system design and implementation. After the designing stage, the implementation process gets start. Classes and their relationships which are developed during the object design; such designs are transformed into a particular programming language at the implementation stage. Generally, the task of converting an object design into source code is a straightforward process. But the feature such as the association of class diagrams in the design model is not possible to directly convert into programming language structures. Actually, most programming languages do not offer any paradigms to implement associations directly. This unit, explores some of the significant features and talks about the several strategies that should be adopted for their implementation.

This unit will explain concepts related to the implementation strategy, such as mapping design to code, and unidirectional and bi-directional associations. Association is a relationship between two separate classes through their objects. The implementation of association is unidirectional or bi-directional. Each association may be either optional, one-to-one, one-to-many associations. This Unit will also illustrate how to transform class definition from Class Diagram. This unit will also cover the important features and strategies for implementing unidirectional or bi-directional associations.

12.1 OBJECTIVES

After going through this unit, you should be able to explain:

- transforming design to code,
- explain different types of associations,
- how to create class definitions from class diagram,
- how the unidirectional associations are implemented,

- how the bidirectional associations are implemented, and
- difference between unidirectional and bidirectional associations.

12.2 MAPPING DESIGN TO CODE

In the software development cycle, when you reach the design part of the software, you create a design document, which includes different UML diagrams such as Design Class Diagram and Interaction Diagram, during the design model. This document is used as input to the code generation in object-oriented programming language.

Now, it is time to use the design document to start writing code. The mapping design to code means that design classes are converted into a program code by writing program statements in a programming language. So, classes in the design model can be implemented by writing program code in an object-oriented programming language such as C++ and Java etc.

A design class represents an abstraction of one or more classes in the software implementation. In fact, class and its objects depend only on the implementation language. For example, in an object-oriented language such as C++ and Java, a class can relate to a plain class. The design model can be very near to the implementation model, depending on how you draw its classes to classes or similar structures in the implementation language. The characteristics of the implementation language impact the design model, you must maintain the class structure as easy to understand and modifiable. The aim of implementation strategy is to use as a general framework for all object-oriented languages. Each programming language has its own number of features and syntax that are difficult to explain in a general process framework. So, you need to focus on the implementation approach so that you may use all potential features and handle the limitations of each programming language.

Mapping Designs to Code is a transformation process from UML class diagrams to class and interface definitions as well as method definitions (you will know more about this in Unit-2 of Block-4). Many UML tools have the facilities to generate code for the programming language. They can generate basic codes but cannot generate your ideas and interpretation of the diagram. Moreover, during object-oriented design modelling, you as a designer try to provide a great base for the code. In an object-oriented language, the implementation stage needs writing source code for class and interface definitions and method definitions.

The next section is related to the creating class definition from the designed class diagram.

12.3 CREATING CLASS DEFINITION FROM CLASS DIAGRAM

In the previous section, you learned to the need to transform a design model into code. This section defines how to transform class definition from a design class diagram.

A design class diagram helps you to write program code for software application development. Class diagram gives you an overview of your application software by depicting classes and their attribute, operations (methods) and relationships (dependencies, Generalization and Associations) among objects. You are aware of the essential elements of the class. This information is enough to create a basic class

definition in an object-oriented language. If you draw your class diagram with the help of UML software tool, it generates the basic class definition from the diagrams. There are many UML-oriented tools for transforming class diagrams into object-oriented code. For example if you are using 'NetBeans' IDE then you may add 'easyUML' plugin into 'NetBeans' to solve your purpose of basic implementation.

The following example illustrates how to create class definitions from the class diagram. Assume that the teacher wants to view the details of the students. Here is a partially created class diagram for Teacher and Student relationship as shown in figure-12.1. The diagram has two classes: Teacher and Student. Class Student defines only two attributes as rollno and name with getDetail() method. On the other side, class Teacher has only one method viewStudent().



Figure 12.1: Class diagram for Teacher and Student relationship

Following are the basic class definitions with attributes and method signatures of the above class diagram:

```

class Student
{
    String rollno;
    String name;
    public student(String rollno, String name)
    {
        this.rollno = rollno;
        this.name = name;
    }
    public String getDetail()
    {
        return(rollno+name);
    }
}
class Teacher
{
    public void viewStudent() { ..... }
}
  
```

Now, you can add some more java code to the above example to complete the program. You can compile and run this program to check how this program works.

```
import java.util.Scanner;
public class Example
{
    public static void main(String[] args)
    {
        Student s1 = new Student("S123","xyz");
        Teacher t1 = new Teacher();
        t1.viewStudent(s1);
    }
}
class Student
{
    String rollno;
    String name;
    public Student(String rollno, String name)
    {
        this.rollno = rollno;
        this.name = name;
    }
    public String getDetail()
    {
        return(rollno+name);
    }
}
class Teacher
{
    public void viewStudent(student s)
    {
        Scanner scan = new Scanner(System.in);
        System.out.println("Enter Student Roll No");
        String getrollno = scan.nextLine();

        if(getrollno.equals(s.rollno))
        {
            System.out.println(s.getDetail());
        }
    }
}
```

12.4 IMPLEMENTING ASSOCIATIONS

Link and Association in UML representation are used for establishing relationships between objects and classes. A link is a logical or physical connection between two or more objects. For example, Ram *enrol-for* IGNOU University. A link is considered as an instance of an association. Association is a group of links that connect objects from the same classes. For example, students *enrol-for* a University. In programming languages, Association is implemented as a reference model in which one object is referenced from other objects, whereas links are not objects as they cannot be referenced but rely on the objects.

You can say that an association relates one or more object to one or more object of the other side. Associations are generally denoted as continuous lines between the participants' classes in the relationship. As shown in Figure 12.2, association is shown between Bank and Manager classes. An association is described by a name and the role played by objects of a class with respect to the other class and the multiplicity of each role. The multiplicity is specified by looking into how many objects of a class can be related to an object of the other class of the association.

The implementation of association is applied either in *unidirectional* or *bi-directional*. As the name implies that unidirectional is a one-way association, and bi-directional is a two-way association. Each association may be either optional, one-to-one, one-to-many association. For example, a teacher might be associated with a subject course such as one-to-one relationship, but a teacher may be associated with many students in the same course class indicates towards one-to-many relationship. If students in one section might be associated with students in another section of the same course, then it is a many-to-many relationship. In a similar manner, if all the sections of the course are related to a single course, then it is a many-to-one relationship.

The direction of traversal is important in association. Traversal means moving to other objects by following its reference. The direction of traversal is one-way from A to B if A only holds the reference to B. There can also be two-way direction where A and B both hold references to each other. Based on traversal, the association is a one-way or two-way association. The one-way or unidirectional association is traversed in one direction, and two-way or bi-directional association are traversed in both directions. If you wish to implement a unidirectional association, you may consider one point; then you shall maintain the association in only one direction. In the case of the implementation of bi-directional association, the association must be maintained in both directions. You can use pointers or association objects for implementing association. A pointer is an attribute that contains object reference, and association object is a pair of dictionary objects; one object is used for the forward direction and another for the backward direction.

The following example illustrates the concept of implementing association.

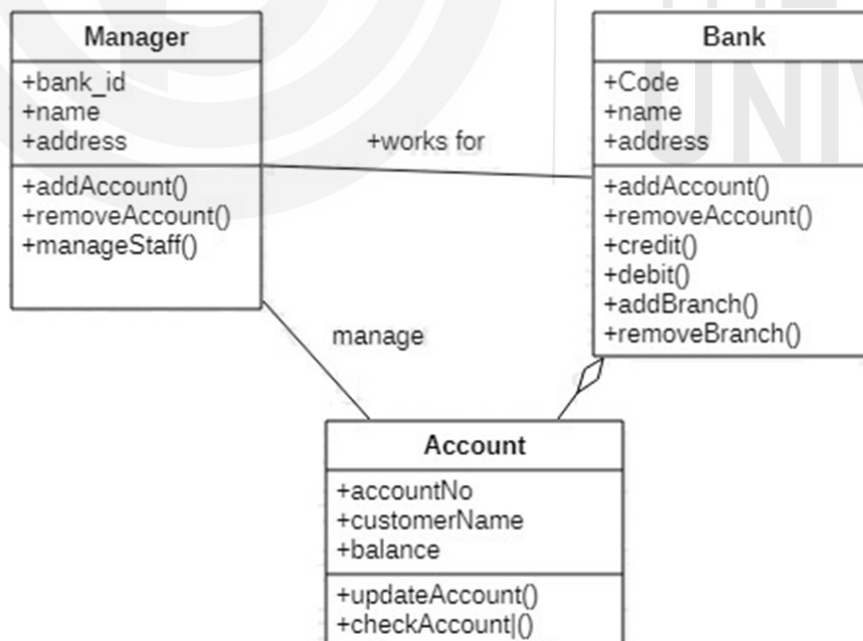


Figure 12.2: Relationship between Manager and Bank

The figure 12.2 defines classes such as Manager, Account, Bank and an association between Manager and Bank classes. Here, the Manager works for Bank and might manage many accounts of people in the Bank.

The implementation of one-way associations must be applied in one direction for the reason that a specific link wishes to be traversed in one direction only. This unidirectional association may be implemented through a single reference, pointing in the direction of traversal.

If one-to-one relationship exists between Bank and Manager, then the simple pointer is used, as shown in Figure-12.3; if there is one-to-many relationship then multiple pointers are used for implementing unidirectional or one-way association.

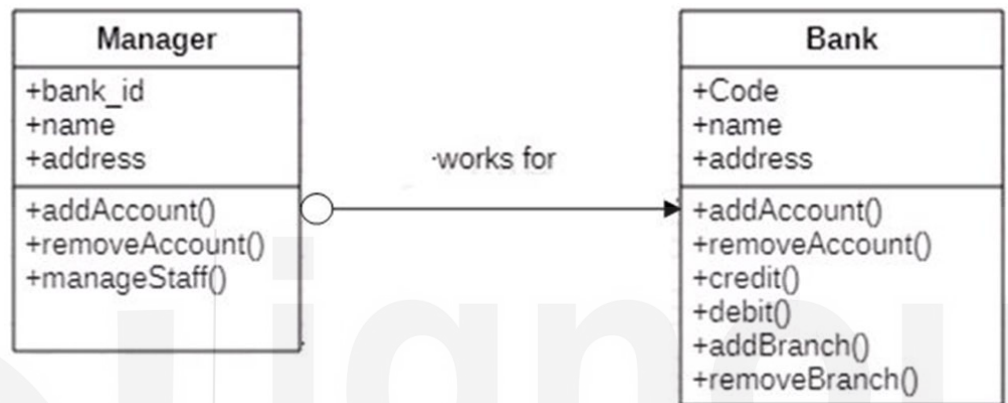


Figure 12.3: Implementation of unidirectional association using pointers

As far as bi-directional associations, they traverse in both directions. Generally, such associations do not traverse with equal frequency in both directions. If bi-directional association traverse with more frequencies in both directions, attributes should be implemented using a set of pointers, as shown in Figure-12.4. This approach provides fast access, but you may have to update attributes in both directions to maintain link consistency. It means that if one attribute is updated on one side, then the attribute on the other side must be updated. If traversal is in less frequency, then implement as an association in one direction only.

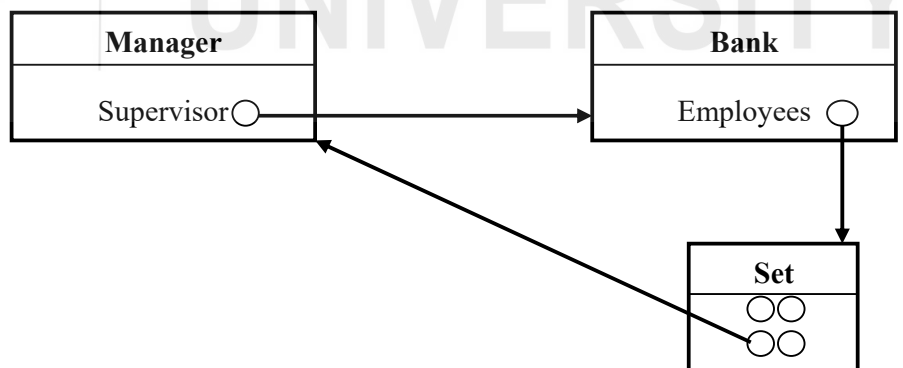


Figure 12.4: Implementation of bi-directional association using pointers

Usually, the two distinct aspects of the implementation of associations are as under:

- The data declaration is essential to define so that it permits the details of actual links to be stored. It consists of defining data members in one class which can store references to objects of the associated class.

- It is required to consider that pointers can be manipulated by the rest of the application. The association's underlying implementation details should be unseen from the client code.

You will study more about the unidirectional implementation in section 12.5 of this unit ,and bi-directional implementation is in section 12.6 of this unit.

Check Your Progress - 1

1. Explain the Mapping of Designs to Code in detail.

2. What is the benefit of designing class diagram?

3. Explain the two distinct aspects of the implementation of associations?

4. What do you understand by the term ‘Association’ in UML? Discuss the types of associations. How does Association differ from Link?

12.5 UNIDIRECTIONAL IMPLEMENTATION

This section will describe cases that help you understand unidirectional implementation of association. This type of association will be supported in one direction only. It means that one class object is linked to another object of associated class in one direction. The direction of traversal of an object can be represented in a class diagram by putting an arrow on the association.

The following subsections define the unidirectional implementation of association for different multiplicity is as follows:

- Optional,

- One-to-one , and
- Association with Multiplicity

12.5.1 Optional Association

In optional association, a link between the participating objects may or may not exist. This type of association is implemented in one direction only. The figure 12.5 shows an optional association between two classes as Person and Credit Card. Every person may or may not have a credit card. It only depends upon the person's need, who desires to take a credit card or not.

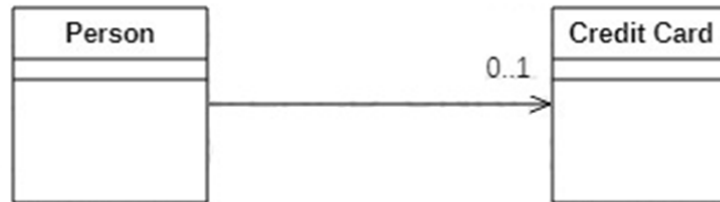


Figure 12.5: Example of Optional Association

The implementation of optional association is easy. You can implement optional association through a simple reference variable, as revealed in the following code. This allows 'Person' object to encompass a reference to one credit card object. If a person is not associated with a card, then the reference variable will be null. The implementation code is listed below:

```
public class Person
{
    private CreditCard cCard

    public CreditCard getCard()
    {
        return cCard;
    }

    public void setCard(CreditCard card)
    {
        cCard = card;
    }

    public void removeCard()
    {
        cCard = null;
    }
}
```

12.5.2 One-to-One Association

The one-to-one association is a simplest form of association to implement between two classes. In one-to-one association, one occurrence of a class is related to exactly one occurrence of the associated class. In unidirectional association, you can traverse from an occurrence of one class to an occurrence of the associated class (as indicated by the direction of the arrow) but not in a reverse direction. This type of association is easily implemented by using an attribute that contains a reference to an occurrence of

the associated class. For example, one-to-one association exists in two classes: Clock and Display, as shown in figure 12.6.

Implementation

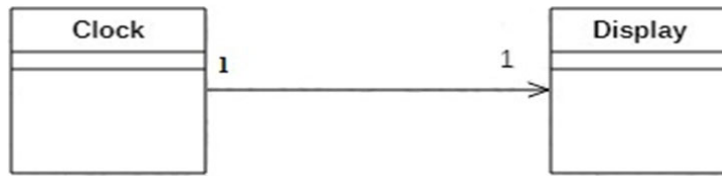


Figure 12.6: Example of One-to-one Unidirectional Association

The implementation of the above figure is as under:

```
public class Clock
{
    private Display theDisplay;
    .....
    .....
}
```

Another example illustrates one-to-one Unidirectional Association. You must know that each person has only one saving account in a bank. So, this is a One-to-one Association. Following figure-12.7 displays one-to-one relationship between Person and Bank. This figure demonstrates two classes: Person and Bank.



Figure 12.7: A One-to-one Unidirectional Association

Each account is associated with exactly one person, and link is traversed from Person class to associated Account class. In this situation, Account is created by the Person constructor. Person class holds a link to the Account Class but not in a reverse direction. Now, you can subsequently write code if the above assumptions are taken into consideration. The implementation of the above figure is as under:

```
public class Person
{
    private Account account;

    public Person()
    {
        account = new Account();
    }
    public Account getAccount()
    {
        return account;
    }
}
```

12.5.3 Association with Multiplicity 'Many'

This type of association exists in those cases where one instance of a class is related to many instances of associated class. You might be seen that every department has many employees. This is an example of one-to-many association between department and employees. In essence, it is a one-to-many association in which data flows from one to many directions.

You may take another example on teacher and student. In a school, each teacher is responsible for teaching a number of students. A teacher object could be associated with many student objects in this association. Here, you can use multiple pointers to implement this association. Each pointer is linked to each student. The teacher class provides operations to maintain the collection of these pointers. Existence of one link must be required between a teacher and student class for implementing this association as shown in figure-12.8. In addition, a data structure is also required for storing all pointers.



Figure 12.8: An association with multiplicity 'many'

This type of association can be implemented using a suitable container class. The class affirms a vector of students as a private data member and the methods to add/remove students from this collection by simply calling the corresponding methods defined in the interface of the vector class.

```
public class Teacher
{
    private Vector stuobj
    public void addStudent(Student stu)
    {
        stuobj.addElement(stu) ;
    }
    public void removeStudent(Student stu)
    {
        stuobj.removeElement(stu) ;
    }
}
```

☛ Check Your Progress - 2

1. Explain the unidirectional relationship? Describe how the one-to-one associations can be implemented.

2. What is an optional association? Explain the implementation of optional association with an example.

3. Explain one-to-one unidirectional association with an example in favour of implementation.

12.6 BI-DIRECTIONAL IMPLEMENTATIONS

In the previous section, you have studied the unidirectional implementation of associations where you can navigate in one direction but cannot navigate in the reverse direction. This section explains bidirectional implementation of association. Bi-directional relationship provides navigational access in both directions, so that you can access the entity of the other side without explicit requests. Links in both directions are required to be maintained for implementing bi-directional associations.

In a bidirectional implementation, each entity has a relationship property or field that refers to the other entity. Through the relationship property or field, an entity of class can access the related object of associated class. If an entity has a related field, the entity is said to “know” about its related object.

A bidirectional association is usually more difficult to maintain than a unidirectional association. It requires additional program code for accurately creating and deleting the relevant objects. This creates a more complex program. Apart from this, an improperly implemented bidirectional association can cause problems for garbage collection of unused objects.

A pair of references are needed for implementing each link in bi-directional implementation of an association. The essential program codes needed to support such implementations are the same as required in the unidirectional implementation. The only variance is that appropriate fields have to be declared in both classes which are taking part in the bi-directional association.

The following subsections define bi-directional implementation of association for different multiplicity is as follows:

- One-to-one and Optional Associations ,
- One-to-many , and
- Immutable Association

12.6.1 One-to-One and Optional Associations

In One-to-one bi-directional association, you can traverse from an occurrence of one class to an occurrence of the associated class and vice versa. The figure-12.9 demonstrates a bidirectional association in which not any arrow is used on any end of the association. You must be able to traverse from an Account class to the corresponding Person class and vice-versa.

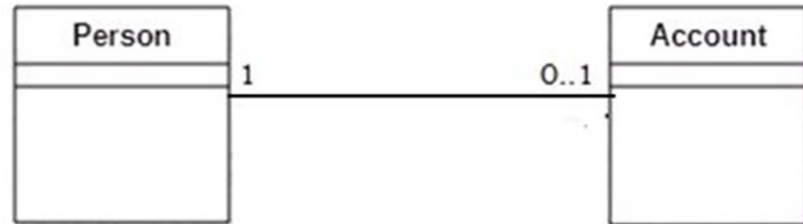


Figure 12.9: Example of one-to-one bi-directional association

For implementation, you have to create two classes: Person and Account. Both the classes contain a link through which it is associated. Each class needs an attribute that contains a reference to an object of the associated class. Each class must have a constructor. The account attribute is initialized in the constructor of the Person class, and it is never modified. In a similar manner, person attribute is initialized in the constructor of the Account class, and it is also never modified. Consider the following implementation code in java for One-to-one bi-directional association.

```
public class Person
{
    private Account account;
    public Person()
    {
        account = new Account(this);
    }
    public Account getAccount()
    {
        return account;
    }
}

public class Account
{
    private Person person;
    public Account(Person person)
    {
        this.person = person;
    }
    public Person getPerson()
    {
        return person;
    }
}
```

12.6.2 One-to-Many Association

In One-To-Many associations, one source is connected with multiple targets. For example, many students can register for one programme. In One-to-many bi-

directional association, you can traverse from an occurrence of one class to many occurrences of the associated class and vice versa.

Implementation

Consider the following example; here we have models two classes: Person and Account. There is a One-to-Many relationship between them, which means that one Person has many Accounts as shown in figure 12.10.

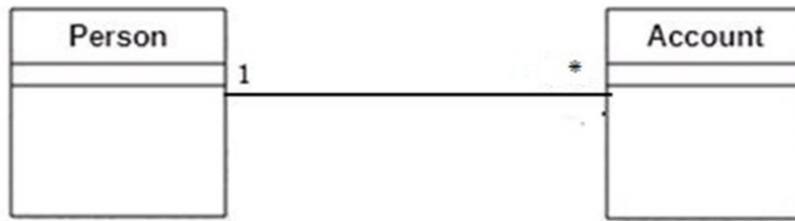


Figure 12.10: A one-to-many bi-directional association

For implementing this association, the Person class can hold a data member to store a set of pointers to Account class. In addition, each account should store a single pointer to a 'Person' class. The responsibility of maintaining the links shall be given to the Person class.

In Object Orientated programming languages, One-to-one relationships are generally implemented with collection objects such as a Set, List or Map, or even a simple array. Java libraries provide collection classes such as HashSet, ArrayList and HashMap, which implement the Set, List and Map interfaces. Following implementation code in java for One-to-many bi-directional association are given:

```
public class Account
{
    private Person person;
    public void setPerson( Person newperson)
    {
        if (person != newperson)
        {
            Person oldperson = newperson;
            person = newperson;
        }
        if (newperson != null)
        {
            newperson.addAccount(this);
        }
        if (oldperson != null)
        {
            oldperson.removeAccount(this);
        }
    }
}

public class Person
{
    private Set accounts;
    public Person()
    {
        accounts = new HashSet();
    }
    public void addAccount(Account ac)
    {
        accounts.add(ac);
        ac.setPerson(this);
    }
    public void removeAccount(Account ac)
    {
        accounts.remove(ac);
        ac.setPerson(null);
    }
}
```

12.6.3 Immutable Association

In object-oriented programming, an immutable object is unchangeable object, and its state cannot be modified after it is created, whereas mutable objects are changeable objects which can be modified after their creation. Immutable objects are inherently thread-safe and offer higher security than mutable objects. Associations are immutable data structures. This means that they carry no state and a copy of an Association is another completely independent Association.

Consider the following example for Immutable Association. An association between account and debit card are traversed in both directions. It is assumed that there is a restriction for the debit card; it can only be issued to person per account. The relevant class diagram which preserves this restriction is shown in figure 12.11.



Figure 12.11: An immutable one-to-one association

Each class defines a data member with reference to an associated class object. The actual code of association is concluded at the time of defining the requirements of the system. The implementation of this association is described as follows:

```
public class Account
{
    private DebitCard DbCard;

    public Account(DebitCard dcard)
    {
        DbCard = dcard;
    }

    public DebitCard getDebitCard ()
    {
        return DbCard;
    }
}

public class DebitCard
{
    private Account theAcc ;

    public DebitCard (Account ac)
```

1. What is bi-directional relationship? How the bi-directional Implementations are made.

2. What is the difference between unidirectional and bidirectional association? Explain an approach to implementing bidirectional association.

3. How is a one-to-one bi-directional association different from one-to-many bi-directional association? Explain how one-to-one bi-directional association is implemented with the help of an example.

12.7 SUMMARY

In this unit, you have learned mainly about implementation strategy, which transforms design into code in a programming language. You have also learned how to create a basic class definition from the design class diagram created during the software development phase. A design class diagram helps you write program code for software application development and gives you an overview of your application software by depicting classes and their attributes, operations and relationships among objects. Each designed class is transformed into one or more classes; it only depends on the programming language.

Association creates a relationship between two separate classes through their objects. This relationship can be unidirectional or bi-directional. This unit also explained the implementing association. There are some decision rules for implementing association, such as in unidirectional association; the decision has been taken into account for maintaining the association in only one direction, while in bi-directional implementations, association must be maintained in both directions. Each association may be either one-to-one, one-to-many or many-to-many relationships. Association represents static relationships between classes.

12.8 SOLUTIONS/ ANSWER TO CHECK YOUR PROGRESS

Check your Progress - 1

1. The mapping design to code means that design classes are converted into a program code by writing program statements in a programming language. Classes in the design model are implemented by writing program code in an object-oriented programming language such as C++ and Java. Mapping Designs to Code is a transformation process from UML class diagrams to class definitions and method definitions. Many UML tools have the facilities to generate code for the programming language. If you draw your class diagram with the help of UML tool, it can generate the basic class definition from the diagrams. There are many UML-oriented tools for transforming class diagrams into object-oriented code.
2. A designed class diagram helps you to write program code for software application development. Class diagrams give you an overview of your application software by depicting classes and their attribute, operations (methods) and relationships (dependencies, Generalization and Associations) among objects. This information is enough to create a basic class definition in an object-oriented language. If you draw your class diagram in a UML tool, it can generate the basic class definition from the diagrams.
3. The two distinct aspects of the implementation of associations are as under:
 - The data declaration is essential to define that it permits the details of actual links to be stored. It consists of defining data members in one class which can store references to objects of the associated class.
 - It is required to consider that the rest of the application can manipulate pointers. The association's underlying implementation details should be unseen from the client code.
4. Association creates a relationship between two separate classes through their Objects. This relationship can be unidirectional or bi-directional. Each association may be either one-to-one, one-to-many and many-to-many relationships.
 - One-to-One: In “one-to-one” relationship, one object is related to exactly one object of associated class. For example, a person can have only one VoterID and this ID can be linked to only one person.
 - One-to-many: A University can have many Students. So it is a “one-to-many” relationship.
 - Many to one: Many students may enrol in a single course which is a “many-to-one” relationship.

Link and Association in UML illustration are used for establishing relationships between objects and classes. A link is a logical or physical connection between two or more objects. For example, Ram enrolls at IGNOU University. A link is considered as an instance of an association. Association is a group of links that connect objects from the same classes. For example, students enrol-for a University. In programming languages, association is implemented as a reference model in which one object is referenced from other object, whereas links are not objects as they cannot be referenced but rely on the objects.

☛ Check your Progress - 2

1. Unidirectional relationship defines only one entity with a relationship property that references the other object. For example, People own book(s). It navigates only in one direction but cannot navigate in the reverse direction.

The association properties can be implemented directly by providing appropriate data declaration in the associated classes. Other semantic features of an association can be set by providing only a limited range of operations in the class's interface or by involving code in the implementation of member functions which confirms that the necessary constraints are maintained.

2. In optional association, a link between the participating objects may or may not exist. This type of association is implemented in one direction only. For example, optional association exist between the person and credit card (see figure-12.5). Every person may or may not have a credit card. It only depends upon the person's need, who desires to take credit card or not. The implementation of optional association is easy. Optional association can be implemented by using a simple reference variable. This permits to person object to contain a reference to one credit card object. If a person is not associated with a card, then the reference variable will be null.
3. The one-to-one association is a simplest form of association to implement between two classes. In One-to-one Association, one occurrence of a class is related to exactly one occurrence of the associated class. In unidirectional association, you can traverse from an occurrence of one class to an occurrence of the associated class but not vice-versa. This type of association is easily implemented by using an attribute that contains a reference to an occurrence of the associated class. For example, one-to-one association is exist in two classes as Clock and Display as shown in following figure.



Figure 12.12 : One to One Association

The implementation of the above figure 12.12 is as under:

```

public class Clock
{
    private Display theDisplay;
    .....
    .....
}
  
```

☛ Check your Progress - 3

1. Each entity in a bidirectional relationship has a relationship property that refers to the associated entity. Through the relationship property, an entity of class can access related objects of associated class. If an entity has a related field, the entity is said to “know” about its related object. A bidirectional

association is usually more difficult to maintain than a unidirectional association. It requires additional program code for accurately creating and deleting the relevant objects. This creates a more complex program. Apart from this, an improperly implemented bidirectional association can cause problems for garbage collection of unused objects. A pair of references are needed for implementing each link in bi-directional implementation of an association. The essential program codes which are needed to support such implementations are the same as required in the unidirectional implementation. The only variance is that appropriate fields have to be declared in both classes which are taking part in the bi-directional association.

2. The bidirectional association provides navigational access in both directions, whereas unidirectional association only provides navigation in one direction. By using bidirectional association, you can access the other side without explicit request, and it also allows you to apply cascading options to both directions.
3. In one-to-one bi-directional association, one occurrence of a class is related to exactly one occurrence of associated class, while in one-to-many bi-directional association, one occurrence of a class is related to many occurrences of the associated class. For example, a teacher might be associated with a subject course (a one-to-one relationship) but also with each student in the same course class (a one-to-many relationship).

12.9 REFERENCES/FURTHER READING

- James Rumbaugh, Michael Blaha, William Premerlam, Frederick Eddy, William Corensen, "Object Oriented Modelling and Design" PHI, New Delhi, 2004.
- <https://www.uml-diagrams.org/index-examples.html>
- <https://www.uml-diagrams.org/class-diagrams-overview.html>
- <https://netbeans.org/features/uml/>
- <http://plugins.netbeans.org/plugin/55435/easyuml>
- <https://www.uml-lab.com/en/uml-lab/tutorials/modellierung-und-codegenerierung/>

UNIT 13 IMPLEMENTATION STRATEGIES - 2

Structure

Page No.

- 13.0 Introduction
- 13.1 Objective
- 13.2 Creating Methods from Collaboration Diagram
- 13.3 Implementing Constraints
- 13.4 Implementing State Charts
- 13.5 Persistency
- 13.6 Summary
- 13.7 Solutions/Answers to Check Your Progress
- 13.8 References/Further Readings

13.0 INTRODUCTION

As you know, object-oriented software development methodology uses three different facets: Object model, Dynamic model and Functional model. The object model defines objects and their relationships in the system; dynamic model describes control structure of objects, and the functional model describes the data transformations of the system. The previous unit of this block introduced you to the conversion of design diagram to executable code, where you learnt the implementation of object model concepts like class and their objects and different associations. This unit is also on the line and gives implementation strategy for the dynamic behavior of object-oriented system. In the next unit, which is the final and last unit of this course, you will learn how to transform object Classes into Tables.

The collaboration diagram is an UML diagram that comes under the Interaction diagram and depicts the relationships between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects. This unit will introduce you to how to create methods from collaboration diagram. In addition to this, you can also learn about the implementation of constraints as well as statecharts. The Constraints represent semantics of object-oriented model elements such as objects, links and attributes. The statechart diagram is used to designate the behavior of a particular class. This unit will cover different statechart implementation strategies such as switch statement, helper object, and collaborator object. At the end of this unit, you will know about the persistency concept.

13.1 OBJECTIVES

After going through this unit you should be able to explain:

- how to create method from collaboration diagram,
- constraints and their implementation,
- statecharts and their implementation, and

- difference between persistence and non-persistence data.

13.2 CREATING METHODS FROM COLLABORATION DIAGRAM

The collaboration diagram is an UML diagram which depicts the relationships between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects. It is basically used to demonstrate how objects interrelate to perform the behaviour of a specific use case. A collaboration diagram carries the similar type of information as a sequence diagram; it focuses on the object structure instead of the chronology of messages passing between them. The sequence and the collaboration diagrams come under the same head as the interaction diagram. The collaboration diagram is also called a communication diagram and is used to represent the object's architecture in the system.

The key purpose of creating this diagram is to illustrate the relationship between objects and works together to perform a particular task. This task is difficult to regulate from a sequence diagram. Besides, collaboration diagrams can also support you in controlling the accuracy of your static model, such as class diagrams.

In an application system, there are one or more collaboration diagrams to find all the eventuality of a complex behaviour. While designing the collaboration diagram, it contains the elements such as actors, objects and their communication links, and messages. Each actor performs a significant role as it invokes the interaction and has a name. The link behaves as a connector between the objects and actors. It depicts a relationship between the objects through which the messages are sent. The messages are exchanged between objects in an order which is represented by sequence numbers. The messages are sent from the sender to the recipient, and the direction must be traversable in that particular direction. Collaboration diagrams are appropriate for analyzing use case diagrams. This diagram can be used to show the dynamic behaviour of a specific use case and describes the role of each object.

You can create collaboration diagrams using StarUML, ArgoUML and many others UML designing software tools. You can also generate collaboration diagram automatically from the sequence diagram. While designing collaboration diagrams, you can follow the basic steps:

- First, you can determine the scope of the collaboration/communication diagram. The scope of a collaboration diagram can be a use case.
- You can put the objects which play a part in the collaboration on the diagram. While putting objects, the most important objects are to be placed in the centre of the diagram.
- If a specific object has a property or retains a state that is important to the collaboration, you can set the initial value of the property or state.
- All objects should be associated with links, and each link should be associated with message(s) or method(s) calls.
- You can add sequence numbers to each message corresponding to the time-ordering of messages in the collaboration.
- You can represent 'if else' condition in a collaboration diagram as shown in figure 13.1.

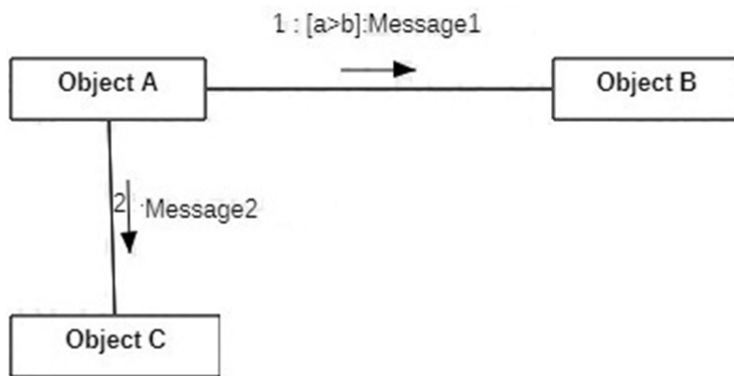


Figure 13.1: Collaboration Diagram showing If-else condition

The implementation of the above diagram is as follows:

```

if (a>b)
{
    ObjectB.Message1();
}
else
{
    ObjectC.Message2();
}
  
```

Let us consider a collaboration diagram for ATM transactions. For this example, first, you create a Use Case diagram and then draw a collaboration diagram. For building a Use Case diagram, you can use the below flow of controls:

- Customer can insert ATM card into the card reader of the ATM console.
- The card reader reads and verifies the card.
- ATM console displays a message and prompts the customer to enter PIN.
- Customer enters PIN.
- ATM console verifies PIN with bank database and confirms the PIN is valid.
- If PIN is **invalid**, ATM console notifies the customer with a message and ejects the card.
- If PIN is **valid**, ATM console represents a prompt with options such as deposit, withdraw and transfer money.
- Customer selects withdraw option.
- ATM console presents prompt to the customer for an accurate amount to be withdrawn.
- The customer enters the amount.
- ATM console verifies with the bank database whether the customer account has sufficient funds. Otherwise, it shows an insufficient funds message.
- ATM console deducts money from customer's account, puts the requested amount in cash dispenser, and prints a receipt.
- ATM console rejects the card and becomes ready for another transaction.

Using the above flow of messages, you can construct the collaboration diagram similar to figure 13.2 where messages/method calls are flowing between objects with

a sequence number. This sequence number specifies the ordering of the messages. The messages appear with pointing arrows from the sender object to the receiver object.

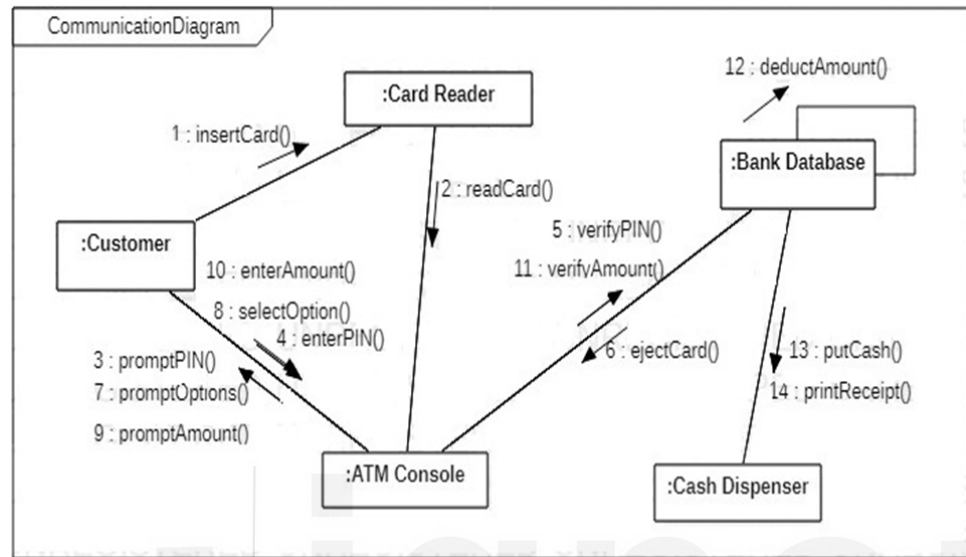


Figure 13.2: Collaboration Diagram for ATM transaction

When you create a collaboration diagram for the real business application , you can create methods as per need. The classes and their methods are created in the Collaboration Diagram given in figure 13.2, which can be summarized in the tabular form:

Table 13.1: Description of Classes and Methods

Class Name	Methods	Class Name	Methods
Customer	insertCard() enterAmount() selectOption() enterPIN()	Card Reader	readCard()
ATM Console	promptPIN() verifyPIN() promptOptions() promptAmount() verifyAmount()	Bank Database	ejectCard() deductAmount() putCash() printReceipt()

Now you can transform methods defined in the collaboration diagram into executable code. You inscribe source code yourself or use software tools such as Eclipse and Concurrent Versions System (CVS). There are number of methods in the figure-13.2, but the pseudocode given below is written for verifyPIN() method only. This method verifies the customer's Personal Identification Number (PIN) using data retrieved from the ATM card's magnetic strip. This method asks the customers to enter their PIN using the keypad of ATM console. If the PIN does not equal to the PIN stored on the card, then the ATM system allows a limited number of repeats. If it is still not matched, then the system invokes ejectCard() method and the card is seized as a

security precaution. If the customer enters the correct PIN then the system invokes promptOptions() method where it provides options such as deposit money, withdraw cash and transfer money. You may try to write code for other methods as well.

```

method verifyPIN
constants no_of_maxpins is 2
variable pin, pin_count is number
set pin_count to zero
read pin from ATMcard
loop until pin_count is equal to no_of_maxpins
input pin from customer
if entered pin matches with cardpin_database
then call promptOptions method
endif
add 1 to pin_count
endloop
if pin_count is equal to no_of_maxpins
then seize customer's card
else call ejectCard method
endif
endmethod// verifyPIN

```

Consider another example of 'Programme Management System'. Using this example, you can learn about the creation of methods from the collaboration diagram. This system contains classes like Admin, ProgrammeCoordinator, Programme, Course and Faculty. Admin initiates the manage programme functionality and interacts with Programme Coordinator. The ProgrammeCoordinator intermingled with different object classes in the system, such as Programme, Course and Faculty. The Programme Coordinator starts the process of creation and modification functionality of the programme. After the programme is either created or modified, ProgrammeCoordinator is responsible for the creation or modification functionality of a course. Each programme consists of many courses. It means that the class Programme and class Course are associated with each other in the form of one-to-many relationships. The ProgrammeCoordinator assigns the course(s) to faculty. Finally, the admin invokes the assigned faculty to the Course functionality, of the Programme. The Collaboration Diagram for Programme Management System is shown in figure 13.3:

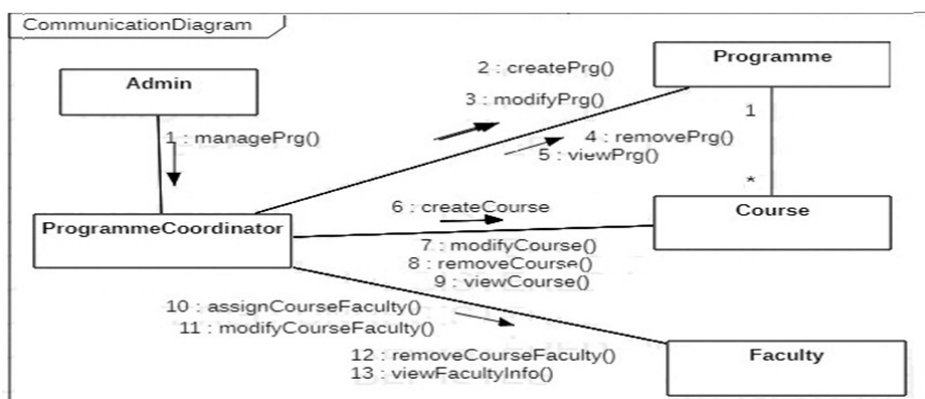


Figure 13.3: Collaboration Diagram for Programme Management System

The collaboration diagram for 'Programme Management System' is shown in figure 13.3. However, the classes are involved in this collaboration diagram for 'programme management system, which contains the following methods. You may write code for the methods given in figure 13.3. List the basic assumptions, then try to write the code in the programming language you are comfortable.

Table 13.2: Description of Classes and Methods in Collaboration Diagram

Class Name	Methods	Class Name	Methods
Admin	managePrg()	Faculty	assignCourseFaculty() modifyCourseFaculty() removeCourseFaculty() viewFacultyInfo()
Programme	createPrg() modifyPrg() removePrg() viewPrg()	Course	createCourse() modifyCourse() removeCourse() viewCourse()

Check Your Progress - 1

1. What is Collaboration Diagram? Why do we need collaboration diagrams?

2. Explain the basic elements that should be considered for defining collaboration diagrams?

3. What is the difference between collaboration and sequence diagram?

13.3 IMPLEMENTING CONSTRAINTS

As you know, if any product is well designed, it will yield fruitful results. In the similar manner, if you want to design a software application with the sound quality of information, then it should be designed with appropriate constraints in the analysis and design phase. If your application is not well designed, it will be erroneous and more expensive to correct. In general terms, constraint means restriction or limitation that is used for optimization purposes. This section describes how to implement constraints in terms of object-oriented paradigms.

A constraint is a condition that restrains the semantics of an element and it must always be true. For example, a person can only get a loan from a bank if the person has an account with the bank. A constraint is either to be declared in the tool and used in several diagrams or identified and applied as needed in a diagram. Some constraints are predefined in UML diagram, and others may be user-defined. When you build a business application, they are closely associated with the constraints. Using the constraints, you can control the software behaviour. You can say that constraint is a rule used for optimisation.

Constraints are used as semantic checks that are implicitly satisfied by a given set of objects. For example, constraints can control the range of an object's integer attribute. You can say that the constraints are defined as rules to be applied to classes and/or fields (attributes) of the class. For example, if a rule dictates that the employee name may never be null, then you would like to place the @NotNull annotation to define the constraint like the following way:

```
public class Employee
{
    @NotNull
    private String empName;

    // ...
}
```

The constraints are used to represent the semantics of object-oriented model elements such as objects, links and attributes. You can use constraints to reflect the rules of the problem domain in the analysis model. Constraints define properties which must be true at run time for the entire system. Usually, constraints do not have names that recognise the contents of their bodies. A constraint is written as a text and appears in a rectangle enclosed with braces with a folded upper-right corner.

You can define constraints in each of the three models of the object-oriented approach. It defines the relationship between the objects of an object model. It also describes the relationship between states and the events of different objects in dynamic modeling, whereas in functional modeling, constraints are used to apply restrictions on the transformations and computations. Constraint contains conditions or restrictions which are included by writing code in an application and verify the current state of the application at run-time. When these constraints are included in the class, it automatically applied to its instances.

You can use a constraint on a single element, such as a class or an association path; the text for constraints may be positioned near the name of the class element and placed in a **note** symbol. A constraint is written as a text string in curly braces using the following syntax:

```
constraint ::= '{' [ name ':' ] boolean-expression '}'
```

Constraints on Objects

The following class diagram demonstrates an example of applying constraints on the attributes. Employee could be as a boss or the worker. Worker's salary cannot exceed the salary of boss. You can see figure 13.4 for the syntax of constraints. It is defined as condition within the braces, placed near the constrained object and then connected with dotted line.

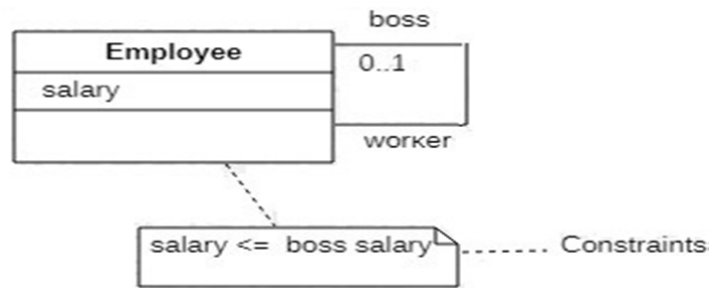


Figure 13.4: Constraints on objects

Constraints on Links

As you know, the multiplicity specifies how many objects of a class can be coupled with an object of the other association class. The multiplicity constraints restrict the number of objects associated with the given object. Object modeling notations define the syntax for displaying multiplicity values like [0, 1] and exactly 1. It has a special notation “{ordered}” which designates the elements of “many” side of an association have an explicit order that must be maintained. The following object model, as shown in figure-13.5, illustrates the Regional Centres of University. Each Regional Centre for each university has a set of employees who have held the regional centre, ordered chronologically.



Figure 13.5: Constraints on Links

Constraints are commonly used for various elements of class diagrams. They work as a bridge to develop functional relations between the entities of an object model. Constraints are helpful to communicate the semantics of classes, validate the correctness of object states at run-time, and repair inconsistent data structures and for various types of reasoning.

The following example of a class diagram represents two classes, Candidate and Job, as shown in figure 13.6. The constraints condition is written near the class which implies on both the classes. The candidates do not apply for the job if the age of the candidate is more than 30.



Figure 13.6: Constraints Example

As yet, you know that the constraint is a condition. The use case works under the pre-, post- and invariant conditions. A precondition states the condition that is required to be met before the use case can proceed. A post-condition specifies the condition which must be true after the execution of the use case. An invariant condition is that condition which is true during the execution of the use case.

Let us consider another class diagram and implementation for constraints, as shown in figure 13.7.

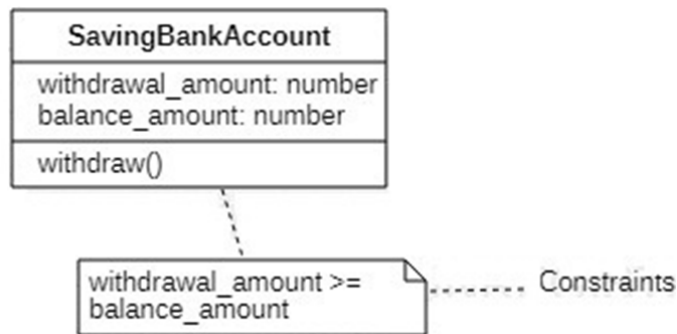


Figure 13.7: Constraints on objects example

The implementation of the above class diagram is as follows:

```

public class SavingBankAccount
{
    public void withdraw(double withdrawal_amount)
    {
        private balance_amount;
        if (withdrawal_amount >= balance_amount)
        { .....
            balance_amount = balance_amount - withdrawal_amount;
        }
        //show error message
    }
}
  
```

There is a number of constraints such as not null, unique, primary key, and foreign key in RDBMS, which you will study in the next unit of this block. A NOT NULL constraint defines a rule which protects a column to not accept null values. The UNIQUE constraint makes sure that all values in a column are different. A PRIMARYKEY constraint is used to identify each row in a table uniquely. A FOREIGN KEY is a primary key of one table that is embedded in another table. You can use primary and foreign key constraints to define the relationship between tables.

When these constraints are added to a relational table, then the table to be tested to confirm that its contents do not oppose the constraint. If the table contents infringe the constraint, the database server returns an error and does not add such constraint.

13.4 IMPLEMENTING STATE CHARTS

In the previous section, you have learnt about the implementation of the constraints. This section describes how to convert statecharts into executable code in object-oriented language like java.

You are already aware that the object model defines objects, attributes and links between them. The attribute values and links detained by objects are known as its **state**. When one object communicates with another, the communication result of these objects is in the form of **events**. Events are external or internal features influencing the system. The state chart or diagram represents the combination of states, events and transition (relationship between two states). This diagram is used in dynamic modeling to define a class's dynamic behaviour. The dynamic model contains multiple statecharts and one for each class.

The following state diagram for booking tickets is shown in figure 13.8 :

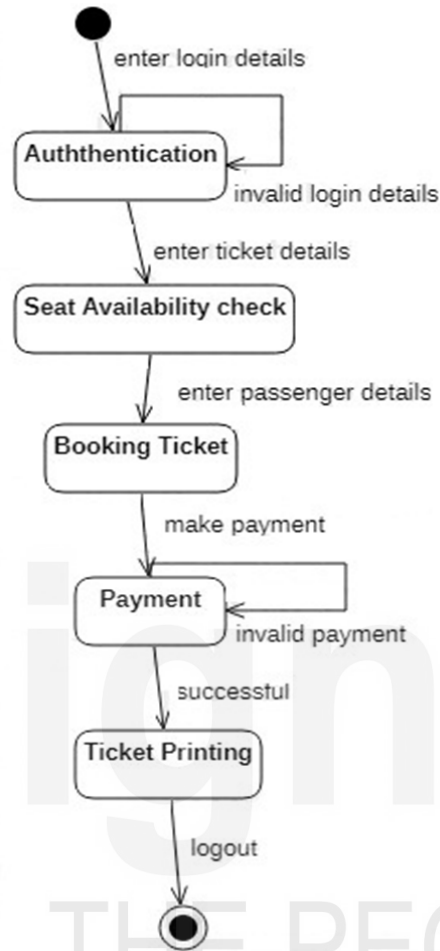


Figure 13.8: State diagram for Booking Tickets

The UML statechart diagram is used to designate the behaviour of a particular class. This diagram can be included only at the class level. A class can have either a statechart or an activity diagram but cannot have both together. While defining the states, you can ignore those attributes that do not affect the object's behaviour. Some classes do not practice statecharts since their objects are uni-state, and they always behave in the same manner.

The simple format of the state diagram is shown in the following figure 13.9:

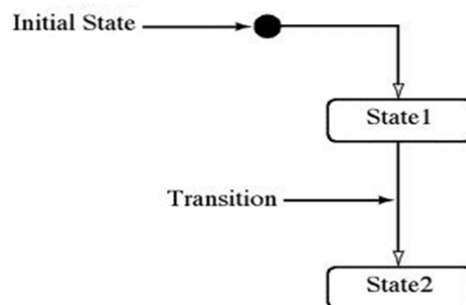


Figure 13.9: Simple Format for State Diagram

There are some strategies for implementation of a statechart diagram such as switch statement, helper object as well as collaboration object.

Implementing Statechart using Switch Statement

This section describes the switch statement approach for implementing statecharts. The states of statecharts diagrams are denoted as objects or scalar variables. Events and actions are indicated as methods.

The most basic technique to implement statechart is the switch statement. You may be aware of the functionality of a switch statement in java where a switch statement executes one statement from multiple case statements depending upon the condition. It is based on the current active state and control transfer to the code for processing the event. It is just like an if-else-if ladder statement. Each case clause of the switch statement denotes a single state of the object and can implement the various actions and activities for the specific state. The actions are implemented as a simple statement. The behaviour of the statechart is placed in a single class. States are denoted as data values. A single scalar variable is a state variable that stores the current active state. The state variable is used inside each event method of the context class. The right case is chosen on the value of the state variable.

This implementation strategy is simple, but it has drawbacks also. In the case of the addition of a new state in statechart, this approach does not allow flexibility in the code. There is code duplicacy in many places, and the reuse of code is very difficult.

Implementing Statechart using Helper Object

Due to the redundant code problem in the switch statement, the notion of a helper object is introduced. This is an alternative object-oriented approach to the switch statement. It places each case clause of a switch statement in a separate object. It provides a natural way to implementation of the statechart diagram. It removes the code redundancy and builds reusable code.

The helper object denotes the current state of the domain object and specifically implements the behaviour of the current state. The domain object transfers all external messages to its helper object, which reacts to the messages on account of the domain object. The helper object signifies only one state of the domain object. The methods of helper objects do not enclose conditional statements and are very simple ways to implement. Whenever changes occur in the state of the domain object, a new helper object is created, which interchanges the older current state and implements the behaviour defined in the new state. The reference to the helper object is recognized as the current state in the domain class.

The statechart shows the behavior of the class, which is a super context or domain class. During this implementation, an abstract state class is created, which describes the interface for capturing the behaviour connected with the states of the statechart. Each state turns into a derived class from the abstract state class. Each transition transforms into a method in the corresponding state class to provide an appropriate way of invoking services on the context object. Each action in statechart will be a method in the context class which contains the reference of the current state in the state object and offers all events for processing to the current state object.

Consider the following statechart diagram shown in figure 13.10 for the video player application with two states: Stop and Play. In the Stop state, when a user presses a play button, then *playButton* event happens, and the device executes the *Playstart* action and goes to 'Play' state. In the Play state, when the user presses a stop button, the *stopButton* event generates, and the device executes the *Playstop* action and goes to the Stop state.

While implementing the statechart as shown in figure 13.10, there are four classes: 'VideoPlayer', 'PlayerState', 'Stop' and 'Play'. The VideoPlayer is super context class and PlayerState is an abstract state class. The two classes such as Stop and Play which

implement the behavior specific to the Stop and Play states respectively and they are subclasses of PlayerState class which offers a common interface for all state classes. These classes are called state classes since they implement individual states of the statechart involved in the domain class.



Figure 13.10: Statechart for a video player

These state classes have a reference to the domain object and name it as 'vPlayer' (as mentioned in the program). In brief, each state in the statechart diagram turns into a class, and each transition from that state becomes a method in the corresponding class, and all actions become methods in the domain class. The domain object forwards the all incoming messages to its current state object, which is mentioned in the program code under the name of *ps*.

The behaviour of the above state diagram can be implemented in Java as follows:

```
class VideoPlayer
{
    PlayerState ps; // helper object
    VideoPlayer()
    {
        ps = new Stop(this);
    }
    void stopButton()
    {
        ps.stopButton();
    }
    void playButton()
    {
        ps.playButton();
    }
    void Playstart() {...}
    void Playstop() {...}
}
//abstract state class
class PlayerState
{
    VideoPlayer vPlayer; //reference to domain object
    PlayerState(VideoPlayer vp) { //constructor
        vPlayer = vp; }
    void stopButton() {}
    void playButton() {}
}
//Stop subclass
class Stop extends PlayerState
{
    Stop(VideoPlayer vp)
    {
        super(vp);
    }
    void playButton()
    {
        vPlayer.Playstart();
        vPlayer.ps = new Play(vPlayer);
    }
}
```

```

    }
}
//Play subclass
class Play extends PlayerState
{
    Play(VideoPlayer vp)
    {
        super(vp);
    }
    void stopButton()
    {
        vPlayer.Playstop();
        vPlayer.ps = new Stop(vPlayer);
    }
}

```

Implementing Statechart using Collaborator Object

Even though helper object provides enhanced remedy than the switch statement for the implementation of statechart diagram, we always believe that there is scope for better improvement in any technique. This section describes the collaborator object approach for implementing statechart diagram.

In the collaboration object approach for implementing statechart, the context class or domain class behaviour is fragmented into context and a state. The context object is an instance of the main class. Using this context object, the client communicates with a collaborator object, which symbolises the behaviour in one of its states.

The collaborator object contains all the state precise actions of the context object. The context object retains a collaborator object which represents to an instance of the current active state object and maintains references of all the state objects. They are created once in the constructor of the context class, and this class is accountable for setting the new state by changing the state reference in the collaborator object. The states are denoted as objects which implement state precise behaviour. The events are signified as methods in the context class. The context object delegates all events to the collaborator object for processing. Unlike helper objects, no new object is created for State Transitions.

An abstract state class is defined, which maintains a reference to access the context class. The statechart diagram contains one or more states, and each state grows as a class which is derived from the abstract state class. You may assign a similar name to the state and the class. The state classes implement the state-specific behaviour. The state object contains attributes and behaviour for a particular state. In this approach, the context object defines the transitions. Each state transition turns into a method in the corresponding state class, providing a way to invoke services on the context object. All the state-specific code resides in one class.

This way, you can implement a statechart in various modes. More you practice more you learn.

Check Your Progress - 2

1. What is a constraint? How do you show constraints on a class diagram?

2. What is statechart diagram? What is the purpose of this diagram? How to implement a statechart into an executable code?

13.5 PERSISTENCY

In general meaning, persistent means preservation. It occurs over a prolonged period. This section describes the persistency in terms of object-oriented approach.

An object resides in a memory space and survives for a specific period of time. In traditional programming, the life of an object is usually the life of the execution of the program that created it. In files or databases, the object life is longer than the duration of the process creating and using the object in a program. This property by which an object stays even after its initiator concludes (close the program) is known as **persistence**. Persistence is important for enterprise applications because it gives required access to relational databases.

Data Persistence

You all know that computer programs operate on data. The most important tasks the application performs are to save and restore data. Persistent data may give useful mechanism for passing data between programs and permits the program to resume processing after a while. If you want data to persist after the program execution, then you must use a permanent data store.

Process Persistence

Persistence represents the lifetime of a process or an object. Referring to the computer threads and processes, a persistent process is a process which cannot be killed or shut down by other processes until the user kills them.

Object Persistence

In connection with data, persistence means an object should not be removed except it is actually erased. This requires appropriate storage and certain procedures which permit the data to persist. You can store persistent data in a persistent database, appearing as long-lasting records or long-lasting objects that fluctuate within devices and software. **Persistent data** is steady and restorable and is accessible after closing the application. This type of data must be saved into a database or internal or external memory, whereas **Non-persistence data** is not accessible after closing the application. The non-persistence data means it is volatile data that is available during the application's execution. For example, relational database management systems store persistent data in the form of records and tables and are accessible even after closing the application.

Identifying Persistent Data

While designing models, models may not always provide a clear picture of the data, and this data should be persistent or transient. There are no limitations to defining permanent data in models, so a single model can bring together persistent and transient data. The **transient data** are temporary in nature and do not have any representation and identifier value in the databases. Transient data is stored in random access memory and valid only inside a program; it is lost whenever the program terminates. **Persistent data** are saved to the database and it survives after transaction updates or program concludes. Generally, persistent data is used to specify the shared, accessed and updated databases across transactions.

Serialization is the process of transforming an object into byte streams to store the object to memory or a database. The serialisation's main objective is to save an object's state so that it recreates when needed. The inverse process is known as **deserialization**, and this process is used to convert byte streams to actual Java objects in memory. The creation of the byte stream is platform-independent. So, the object serialized on one platform can be deserialized on a different platform.

Java provides serialization with the use of 'Serializable' interface. This interface (java.io.Serializable) is a marker interface which does not declare any data members and methods. Using this interface, "mark" the java classes, and objects of these classes may get a certain ability. When it is done, the methods 'writeObject' of ObjectOutputStream classes used for serializing an Object and 'readObject' of ObjectInputStream class is used for deserializing an object. This way, persistence can be implemented in java.

The next unit will explain to you how to map Objects to tables in the database.

Check Your Progress - 3

1. What is the difference between persistent and transient objects?

2. What do you understand by Data Persistence? How you will make your data persistent?

3. What is the difference between persistent data and non-persistent data?

4. What is serialization? Where it is used, and why?

13.6 SUMMARY

This unit discussed the collaboration diagram; implementing constraints and statechart in object oriented paradigms. The key purpose of creating collaboration diagram is to illustrating the relationship between objects and to explain how objects work together

to perform a particular task. It is also called a communication diagram. This unit has also covered about the constraints, which is useful for controlling the software behaviour. A Constraint is a rule that is used for optimization purpose.

Further you have been explained the implementation of behavioral features of model through the statechart. The dynamic model contains multiple state charts and one for each class. Some of the statechart implementation techniques are also discussed, such as switch statement, helper object as well as collaborator object. The switch statement is the most basic and earliest technique to implement a statechart. Due to the redundant code problem in switch statement, an alternative object-oriented approach as helper object approach is introduced for implementing statecharts. One more improved approach as collaborator object has also been introduced to implement statechart diagrams.

Finally, persistency concepts have been discussed concerning object oriented databases and relational databases. The Persistence data is accessible after closing the application. This data must be saved into a database or internal or external memory. Non-persistence data is not available after closing the application. The non-persistence data means it is volatile data that is available during the application's execution. You have also known about serialization, which is transforming an object into byte streams to store the object in memory, a database or a file.

13.7 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

Check Your Progress - 1

1. The collaboration diagram is a UML diagram which depicts the relationship between the objects in a system. It defines a functionality of the system by demonstrating a set of chronological interactions between objects. It is used to demonstrate how objects interact to perform the behaviour of a use case. The purpose of collaboration diagrams is to visualize the interactive behaviour of the objects in a system. It is used to capture the dynamic behaviour of the system.
2. Four basic elements are used to construct the collaboration diagram: objects, actors, links and messages. Each actor performs a significant role as it invokes the interaction and has a name. The link behaves as a connector between the objects and actors. It depicts a relationship between the objects through which the messages are sent. The messages are exchanged between objects and include sequence numbers.
3. The difference between collaboration and sequence diagrams are as follows:
 - The collaboration diagram is used to depict the relationship between the objects in a system. The sequence diagram is used to visualize the sequence of calls in a system which is used to implement a precise functionality.
 - The collaboration diagram is used to denote the system's structural architecture and the messages sent and received. The sequence diagram is used to signify the sequence of messages that are transmitted from one object to another.
 - The collaboration diagram focuses on the object's interaction, whereas the sequence diagram concentrates on the time sequence.

Check Your Progress - 2

1. A constraint is a condition that restrains the semantics of an element, and it must always be true. For example, a person can only get a loan from a bank if the person has an account with the bank. Some constraints are predefined in

UML diagram, and others may be user-defined. The constraints are used to represent the semantics of object-oriented model elements such as objects, links and attributes. A constraint is written as a text and appears in a rectangle enclosed with braces with a folded upper-right corner. You can use a constraint on a single element, such as a class or an association path; the constraint text may be positioned near the name of the element and placed in a note symbol.

2. A state shows a graphical depiction of the status of an object. It usually reflects a specific set(s) of its attributes and their relationship. A statechart transition consists of a single arrow between a source and a destination and shows a relationship between two states. Transitions denote the reaction to a message in a given state.

Statecharts are used to describe the run-time behaviour of objects of a class. Each statechart is included at a class level which describes all behavioural features of the objects in that class. Some classes do not use state charts because their objects are uni-state and always behave similarly.

Check Your Progress - 3

1. The difference between transient objects and persistent objects are as follows:
Persistent object is an instance of a class in the domain model that represents its values in databases, whereas a transient object is an instance of a class in the domain model which is created in memory. The garbage collector will remove the transient objects if they don't have any use and reference in the database. The persistent objects are saved in the databases. For example, if you acquire an entity from a database, then that entity is persistent. When you create a new entity, it is transient until it is stored in a database. The persistence in Object Oriented Databases is handled by creating Persistent Objects and Transient Objects.
2. Persistent data is data which has a longer lifetime than the program that created it. Enabling data to be stored on a permanent storage medium provides persistency. The most common techniques used are storing data in the form of files or making use of a back-end database system.
3. The Persistence data is accessible after closing the application. This type of data must be saved into a database or internal or external memory.
Non-persistence data is not accessible after closing the application. It means that it is volatile data available during the application execution only.
4. Serialization provides an appropriate and straightforward approach of making data persistent. Serialization transforms an object into byte streams to store the object in memory or a database. It is most suitable when the amount of data included is relatively small. If more amounts of data are to be stored, serialization may no longer be an appropriate approach for storage.

13.8 REFERENCES/FURTHER READINGS

- Kenneth Barclay and John Savage, “ Object-Oriented Design with UML and Java”, 2003.
- James Rambaugh, Michael Blaha, William Premerlam, Frederick Eddy, and William Corensen, “Object Oriented Modelling and Design” PHI, New Delhi, 2004.
- <https://www.uml-diagrams.org/communication-diagrams.html>
- <https://www.uml-diagrams.org/communication-diagrams-examples.html>
- <https://www.uml-diagrams.org/constraint.html>

UNIT 14 OBJECTS MAPPING WITH DATABASES

Structure

Page No.

- 14.0 Introduction
- 14.1 Objectives
- 14.2 Relational Database Schema for Object Models
 - 14.2.1 General DBMS Concepts
 - 14.2.2 Relational DBMS Concepts
 - 14.2.3 RDBMS Logical Data Structure
- 14.3 Object Classes to Database Tables
- 14.4 Extended Three Schema Architecture for Object Models
 - 14.4.1 The use of Object IDs
 - 14.4.2 Mapping Object Classes to Tables
- 14.5 Mapping Associations to Tables
 - 14.5.1 Mapping Binary Associations to Tables
 - 14.5.2 Mapping Many-to-Many Association to Tables
 - 14.5.3 Mapping Ternary Associations to Tables
- 14.6 Mapping Generalizations to Tables
- 14.7 Interfacing to Databases
- 14.8 Summary
- 14.9 Solutions/Answers to Check Your Progress
- 14.10 References/Further Reading

14.0 INTRODUCTION

In the new era, the project development team uses Object Oriented Technology such as Java or C# to build their business application software and store the data in a relational database. The relational databases are widely used database software in organizations. This unit introduces you to how to implement the object model in the relational model. This unit starts with some introductory concepts on general database as well as relational DBMS concepts. As you have already got updated with the knowledge of Object Modeling in the previous blocks of this course. This unit describes how object modeling relates to the three schema architecture. This schema architecture defines how the data is represented or viewed by the user in the database. In addition to these concepts, this unit explains the mapping of Object Classes to Tables. You have learnt about the associations and generalizations in an earlier block of this course. This unit will revisit how to map different associations such as one-to-one, one-to-many, many-to-many, and ternary association as well as generalizations to database tables. At the end of this unit, you will know about interfacing with the database.

14.1 OBJECTIVES

After going through this unit, you should be able to explain:

- basic relational DBMS concepts,
- how to map object classes to database tables,
- extended three schema Architecture for Object Models,
- how the associations are mapped to the tables, and

- how the generalizations are mapped to the tables.

14.2 RELATIONAL DATABASE SCHEMA FOR OBJECT MODELS

As you know, a database is a group of interconnected data files with structures. It is designed to come across the various information necessities of the organization. The database is created by using DBMS software. The designing of the database is the most significant task for database application software. The term relation schema describes the outline of the database. Relation schema expresses the design and structure of the relation (table), such as table name, set of attributes or field names. This section provides you with a small description of the introductory lesson related to the database concepts.

14.2.1 General DBMS Concepts

A DBMS is a software program which is used for creating, retrieving and controlling access to permanent data. There are various motives for using DBMS, such as Integrity, Security, Crash Recovery, Data Distribution and Sharing between users as well as applications.

To effectively manage data using a DBMS, you may know certain **key terms**:

Database Schema

Database Schema is the design of a database. It is the frame of the database that signifies the structure (table names and their fields/columns name). Each column can hold which type of data, constraints on the data to be stored (if any) and the relationships between the tables. You can also say that the database schema is a logical structure of the database and tells us how the data are organized in a database.

Data Constraints

Sometimes you want to place certain restrictions on the type of data that can be inserted in column(s) of a table. This is done by defining one or more constraints on that column(s) while creating the tables. In other words you can say that it is a condition which applicable on the column(s). It ensures database integrity and a few names of the constraints such as Unique, Primary, Default and Check constraint.

Data Dictionary

The database schema along with numerous constraints on the data is stored in a database catalog or dictionary. It is called meta-data. A meta-data is data about the data.

Database Design

The database design is an important and tough task for any business database application. Database design describes the database structure used for creating, storing and maintaining information. The main objective of designing a database is to build physical as well as logical models of the designs for the planned database system. There are two methods to design a database i.e. **attribute driven design** (ADD) and **entity-driven design** (EDD). In attribute driven design, assemble a list of attributes related to the application and normalize the sets of attributes that preserve functional dependencies (FD). In contrast, in the Entity driven design method, define entities which are significant to the application and explain them. Usually, database design has limited entities as compared to the attributes. Entity design is more considerable due to being easily manageable. Object modeling is a kind of entity design.

The three schema architecture provides an abstract view of data to users. The three schema architecture is shown in figure-14.1. The database design contains three layers such as external, conceptual and internal schemas. The **external schema** is external view which represents some portion of the entire database. It is a higher order abstraction which is seen by the user. The next level is **conceptual view** which represents data logically. It defines what data is stored really in the database. The third level of this architecture is an **internal schema** which shows the lowest level of abstraction and closest to physical storage. Object modeling is suitable for designing both the external and conceptual schema. For this, you should construct one object model for each external schema as well as another object model for the conceptual schema.

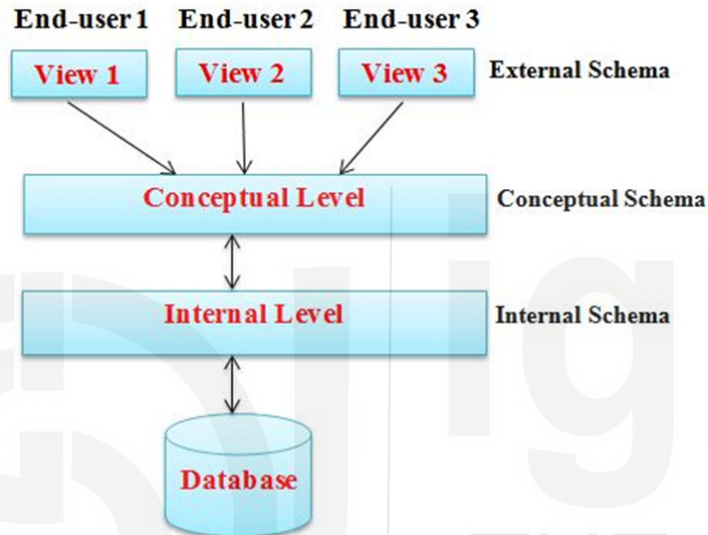


Figure 14.1: The three schema architecture

14.2.2 RDBMS Logical Data Structure

A relational DBMS (RDBMS) is a database management computer program based on a relational data model. Edgar F. Codd invents it. The relational model can store data with one or more relations (tables). A **relation** is a two-dimensional table i.e. row and column. In relations (tables), data is stored in the form of **tuples** (rows) and columns. A **row** is the horizontal portion of the table. One row denotes one **record** of the table. Each row in a table is identified by a **primary key** that does not permit duplicate row. Columns of a table are called as field names or attributes. Column is the vertical portion of the table. The numbers of attributes (columns) in a table is known as **degree** of the relation (table). The number of rows (tuples) in a table is known as **cardinality** of the relation.

14.2.3 Relational DBMS Concepts

In addition to what has been discussed in section 14.2.2, there are more relational DBMS concepts which are discussed below:

RDBMS Operators/Commands

The Structured Query Language (SQL) is a popular language for RDBMS that enables you to create and manipulate a relational database. SQL provides many operators for manipulating tables. SQL has many processing capabilities, such as Data Definition Language (DDL), and Data Manipulation Language (DML). The SQL DDL provides

commands for creating and deleting tables, creating indexes and modifying relation schemas, whereas DML provides commands for data retrieval, data deletion and modification of data stored in the database. Using DML, you can also insert new data into the database. For example, the SQL SELECT command is used to retrieve data from one or more tables. The syntax of the SELECT command is like the following:

```
SELECT <column name> [, <column name>, ...]
FROM <table name> [WHERE <condition>]
```

RDBMS Integrity

For maintaining the quality of the information in the database, relational DBMS provides integrity rules. Data integrity means that the data is accurate and consistent in the database. There are two rules of integrity in Codd's model such as entity integrity and referential integrity.

Integrity Rule 1(Entity integrity): It defines that each table have exactly one primary key. A primary key is a set of one or more attributes that can uniquely identify each row in a relation.

Integrity Rule 2 (Referential integrity): It needs that a foreign key must have a matching primary key. Referential integrity constraint ensures that relationships between records in related tables are valid.

A foreign key is a primary key of one table that is embedded in other table. The primary, as well as foreign keys, are shown in the following tables.

Course Table:

CourseCode	CourseName
MCS-218	Data Communication and Computer Networks
MCS-219	Object Oriented Analysis and Design
MCS-220	Web Technologies

Student Table:

RollNo	Name	Age	CourseCode
1	Ram	20	MCS-218
2	Mohan	25	MCS-219
3	Govind	30	MCS-220
4	Narayan	28	MCS-218

The above example shows Primary and foreign keys. In this example, there are two tables: Student and Course. **CourseCode** is the primary key of the Course table, and **RollNo** is the primary key of the Student table. **CourseCode** is a foreign key in the Student table. It would not be acceptable to change Mohan CourseCode to MCS-221 because MCS-221 is not defined in the Course table. The foreign key is used to create a relationship between two tables.

Normal Forms

Database design is difficult task. Normalization theory is a valuable support in the database design process. Normalization is a data analysis process. While designing the databases, there are unnecessary repetitions of data. The normalization is used to reduce the repetition of data. The normalisation process is related to transforming the conceptual schema (logical data structure) into a computer-representable form. It is used to avoid logical inconsistencies from table update operations. There are various levels of normal form, such as 1NF to 5NF and BCNF.

A table is in **first normal form** (1NF) if and only if all underlying domains of the table contain atomic values. A table is in **second normal form** (2NF) when it satisfies the first normal form and every non-key attribute fully depends on the primary key. A table is in the **third normal form** (3NF) when it satisfies the second normal form, and every non-key attribute is non-transitively dependent on the primary key.

View

View is a virtual table whose contents are derived from one or more tables depending upon the conditions. A view does not physically store data in the database. It is used just like any other table. A view is a query like the **SELECT FROM ...clause** that works on the physical table. The purpose of using view is for data security; it permits a user to use the data through the view only, and hidden data is not available to the user. DML operations such as INSERT, DELETE and UPDATE can be applied on the views.

Check Your Progress - 1

1. What are the different integrity constraints in RDBMS?

2. What is database design in a relational database management system? Explain two methods of database design.

3. What are the three levels of schema architecture?

14.3 OBJECT CLASSES TO DATABASE TABLES

In the previous section, you have gone through the general DBMS and relational DBMS concepts that are most useful during the database design process. This section provides you with how to transform an object model into a relational database. This section focuses on extended three schema architecture for the object models, use of object IDs, and mapping object classes to tables.

14.4 EXTENDED THREE SCHEMA ARCHITECTURE FOR OBJECT MODELS

This section moves towards the concepts of relational database design as RDBMS technology, which is today's more suitable technology for business applications. It describes how object modeling relates to the three schema architecture.

The three schema architecture is used for designing the structure of database systems. This architecture is used to separate the user applications and physical databases. The three schema architecture covers three levels: External, Conceptual and Internal.

External Schema: It designates high-level data model and closest to the user.

Conceptual Schema: It defines how the database looks in front of users.

Internal Schema: It defines the physical storage structure of the database. The following figure-14.2 illustrates how object modeling connects to the three schema architecture.

In the first step toward this design, you should develop object models for the external and conceptual schema, and after this, you should transform each object model into table model. Views and interface programs which are connected with external tables convert to conceptual tables. Conceptual tables transform these into the internal schema.

The **object model** defines the static structure of the objects in a system and their relationships. It emphasizes on logical data structures. Each object model contains classes, associations, generalizations and so on. The goal of constructing an object model is to grasp those ideas from the real world which are significant to an application.

You must use any mapping procedure when transforming the object model to the table model. There are many mapping alternatives, and they comprise mapping rules. For example, if you want to map association(s) to the table(s) then there are two alternatives for mapping association. Similarly, when you transform the generalizations to relational tables, you can use different strategies. You will study mapping associations to tables in section 14.5 and mapping generalizations to tables in section 14.6 of this unit. During transformation, you can also insert some points/keys into the table which are missing in the object model, such as the primary key and candidate keys for each table and foreign key wherever require, also put some constraints on the attributes whether each attribute can be **null**, or **not null**. Those attributes are defined as candidate keys and generally should not be null. You must assign a domain to each attribute and list groups of attributes that are frequently accessed.

The **internal schema** of the three schema architecture contains SQL commands which are used to create the tables, attributes, assign constraints, and keys. The latest database software version comes with a wizard facility that automatically creates tables and indexes. While designing a relational table, you should put restrictions on the length and legal characters for names. You can see the conversion process of the external table model to the conceptual table model to the internal schema in the rounded part of the figure 14.2.

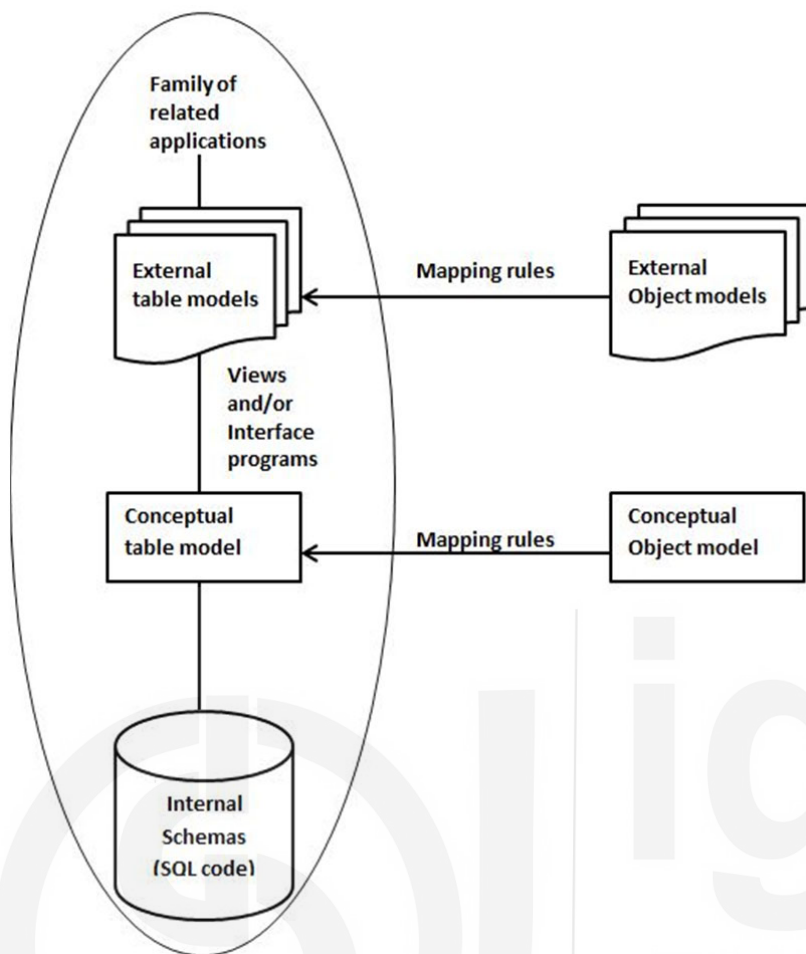


Figure 14.2: Object modeling and the three schema architecture for RDBMS

14.4.1 The use of Object IDs

An object identifier (OID) is a unique name for any type of object or entity. It is a mechanism to identify an entity. When you derive a table from the class, each table has an ID for the primary key. When you derive tables from association(s), then one or more object IDs collectively form the primary key. An ID is equal to the database concept. In a database, an object ID is a set of integers which uniquely identify each row (or record) in a table.

There are some advantages and disadvantages of using IDs. The benefit is that it does not make any changes to the data. The stability of IDs is mainly important for associations as they refer to objects. Any changes in the name of IDs, will update many associations. So, IDs give an unvarying mechanism for referencing objects. The disadvantage is that RDBMS does not support the IDs. Actually, there is no concept for generating the IDs in RDBMS because it emphasizes the data located and manipulated based on attribute values.

You can define IDs as attributes for your tables and accept a mechanism for handling them as per the relational database you have adopted. You cannot use IDs in applications where you directly access the database.

14.4.2 Mapping Object Classes to Tables

When you map class objects to the relational model, you shall start with the attributes of a class. These attributes will translate to one or more columns in a table of

relational database because some of the attributes are used for temporary calculations. For example, an employee class may have a *grossPay* attribute that is used for some calculation. It means that all attributes will not be transformed to tables or in persistent storage. While mapping objects to tables, objects in a class may be segregated horizontally and/or vertically for ease of access. For example, if you have a class with many attributes but few are often used, then you can segregate frequently accessed objects in a table and the remaining objects in another table using horizontal partitioning. In a similar manner, if a class has attributes with different access forms, then partition the objects vertically. Both types of partitioning are useful for improving efficiency. The following figure-14.3 illustrates the vertical and horizontal partitioning of the tables.

Vertical Partition	StudentID	Name	StudentID	Address
	1	Ram	1	R.K. Puram, Delhi
	2	Shyam	2	Papum Para, Itanagar
	3	Govind	3	Jalukbari, Gawahati

Horizontal Partition	StudentID	Name	Address
	1	Ram	R.K. Puram, Delhi
	2	Shyam	Papum Para, Itanagar

StudentID	Name	Address
3	Govind	Jalukbari, Gawahati

Figure 14.3: Vertical and horizontal partition of tables

You can see in the figure14.3 where an object class is converted into a table. Class 'Student' has attributes such as Name and Address. You can list these attributes in the table model and insert the implicit object ID into it. During making a table model, you can mention more details. You can specify StudentID, which cannot be null because it is a candidate key. The name attribute cannot be null, a name must be entered for every student. You cannot define name of student as candidate key because two students may have the same name, but the address attribute may be null. You can also assign a primary key to each table. The Structured Query Language is used to create a Student table. The following figure 14.4 demonstrates you how to map object class to a table.

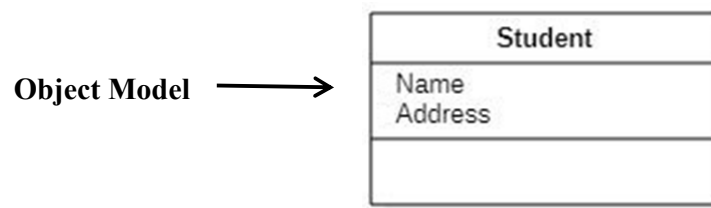


Table Model →

Attribute Name	Null	Domain
StudentID	N	ID
Name	N	Name
Address	Y	address

Figure 14.4: Mapping Object Class to Table

Creating Student table:

SQL Code →

```
CREATE TABLE Student (
  StudentID ID not null,
  Name varchar(30) not null,
  Address varchar(30),
  PRIMARY KEY (StudentID) );
```

☞ Check Your Progress - 2

1. What is an object IDs? What is the advantage of using object IDs?

2. Explain how the object classes are mapped to database tables.

3. Explain an object model with a simple example model and transform it to the relational table.

14.5 MAPPING ASSOCIATIONS TO TABLES

As you already know, association is a relationship between two separate classes through their objects. Associations are generally denoted as continuous lines between the participants' classes in the relationship. An association may be one-to-one, one-to-many, many-to-many or a ternary association. The association mapping to tables depends on the association type and multiplicity of the association. This section will discuss about the mapping of different types of associations into database tables.

14.5.1 Mapping Binary Associations to Tables

This section describes the mapping of one-to-one association and one-to-many association to the database tables.

In **one-to-one Association**, one occurrence of a class is related to exactly one occurrence of the associated class. For example, the following class diagram shows one-to-one association between Country and City. In the figure 14.5, relationship type *has_capital* shows how each Country holds exactly one City.



Figure 14.5: Object model for one-to-one Association

While mapping such a relationship to the tables, the primary key of one table is included in another table, which means that you can handle one-to-one association by using Foreign Key. Like in the above example, *CountryCode* is a primary key of the Country table. *CityCode* is a primary key of the City table. The foreign key that is primary key of the Country table is inserted into the City table.

The figure 14.6 is shown as a SQL code for the above one-to-one association example. While mapping to tables, first, you create a table model or list the attributes as you have learnt in section 3.3.3 as mapping object classes to tables; after that, you may create SQL code for the table. The following SQL code (figure 14.6) is generated for SQL Server database.

```

Create table country
( countrycode int not null,
  countryName varchar(30)not null,
  Primary key(countrycode));

Create table city
( CityCode int not null,
  CityName varchar(30)not null,
  countrycode int not null,
  Primary key(CityCode),
  Foreign key (countrycode) References country );
  
```

Figure 14.6: SQL code for mapping one-to-one association to tables

You may also merge the above example (one-to-one association) into a table and store both objects and association all in one table. This approach is useful for improving performance and reducing database storage. But you should be careful while using this approach as it may introduce redundancy.

In **one-to-many association**, one occurrence of a class is related to many occurrence of the associated class. For example, the class diagram for one-to-many association between University and Employee classes is shown in the figure 14.7.

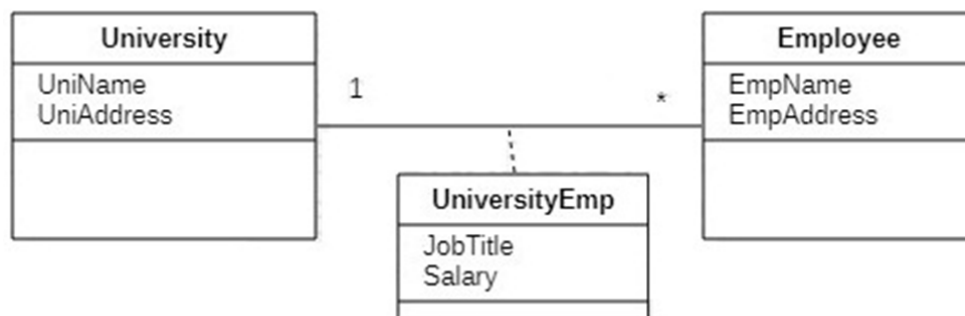


Figure 14.7: Object model for one-to-many Association

When you're mapping an one-to-many association to tables, you can have two options; you may use a **foreign key** in the table for 'many' class or create a **distinct table** for each association class. Using the first option, you can create a table model from the object model, as shown in figure 14.7.

The following figure 14.8 is shown the table model for the object model, which is defined in figure 14.7.

University Table Model

Attribute Name	Null	Domain
UniID	N	ID
UniName	N	Name
UniAddress	Y	address

Employee Table Model

Attribute Name	Null	Domain
EmpID	N	ID
EmpName	N	Name
EmpAddress	Y	Address
UniID	N	ID
Job title	Y	Title
Salary	Y	Salary

Figure 14.8: Table model for one-to-many Association(using foreign key)

You can see the figure-14.8, there are two tables as University and Employee tables. The *UniID* is the primary key of the University table, and *EmpID* is the primary key of the Employee table. The foreign key that is the primary key of the University table is inserted into the Employee table. You can create relational tables using the table model defined in figure-14.8. The SQL code generation of this example is not shown here, and you may create this easily with the help of other examples.

While mapping to tables, first, you can create a table model and then write SQL code for association. Now using the second option, you may create a distinct table for one-to-many association. Here, take the above example of the object model, which is shown in figure 14.7 for transforming to a table model.

University Table Model

Attribute Name	Null	Domain
UniID	N	ID
UniName	N	Name
UniAddress	Y	address

Employee Table Model

Attribute Name	Null	Domain
EmpID	N	ID
EmpName	N	Name
EmpAddress	Y	Address

UniversityEmp

Attribute Name	Null	Domain
UniID	N	ID
EmpId	N	ID
Job title	Y	Title
Salary	Y	Salary

Figure 14.9: Table model for one-to-many association – using distinct table

In figure 14.9, you can see the table model for the object model defined in figure 14.7. This table model consists of three tables: University, Employee and UniversityEmp table. Here, all the classes in the association are mapped to a distinct table. The *UniID* is the primary key of the University table, and *EmpID* is the primary key of the Employee table. The attributes *UniID* and *EmpID* join together to form the only candidate key for the *UniversityEmp* table. Now, you can create relational tables like the following SQL code in figure 14.10.

```
Create table University
( UniID varchar(10) not null,
  UniName varchar(30) not null,
  UniAddress varchar(30),
  Primary key (UniID));

Create table Employee
( EmpID varchar(10) not null,
  EmpName varchar(30) not null,
  EmpAddress varchar(30),
  Primary key (EmpID));

Create table UniversityEmp
( UniID varchar(10) not null,
  EmpID varchar(10) not null,
  Job_title varchar(30),
  Salary int
  Primary key (UniID, EmpID),
  Foreign key (UniID) References University,
  Foreign key (EmpID) References Employee);
```

Figure 14.10: SQL code for the one-to-many Association (using distinct table)

14.5.2 Mapping Many-to-Many Association to Tables

In many-to-many association, M occurrence of a class is in a relationship with N occurrence of the associated class, as shown in the following figure 14.11.

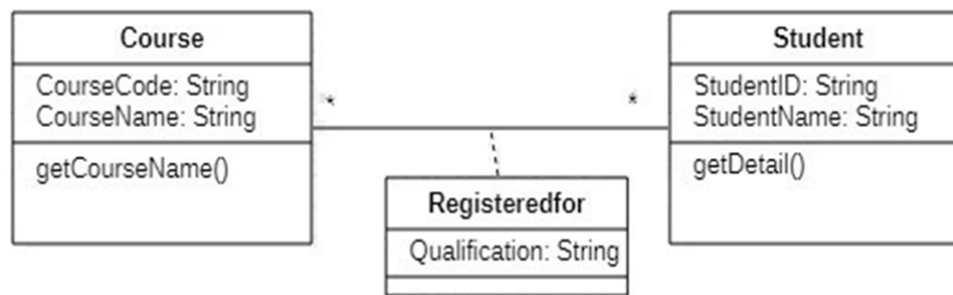


Figure 14.11: Object model for many-to-many Association

While mapping such relationship to the tables, you require an associative table that references the primary keys of the associated records. The name of this associative table is usually either the combination of the names of the tables that it associates or the name of the association that it implements. If you need to store any extra information for this relationship, you can add columns to the associative table. Let's see an example of such a relationship in the class diagram (figure-14.11). This example contains two classes, Course and Student, and one associated class. The relationship type *Registeredfor* shows how each course has M student registered for it whereas depicted other, each student Registered for N different courses. Now you can create a table model by taking the help of the above examples described in this unit.

A many-to-many association always maps to a distinct table. The primary keys for both related classes and any link attributes become attributes of the association table. The attributes (classes i.e. Course and Student) *CourseCode* and *StudentID* join together to form the only candidate key for the *Registeredfor* table.

The following is a SQL code for the above many-to-many association mapping to tables.

```

Create table Course
( CourseCode varchar(10) not null,
  CourseName varchar(30) not null,
  Primary key(CourseCode));

Create table Student
( StudentID varchar(10) not null,
  StudentName varchar(30) not null,
  Primary key(StudentID));

Create table Registeredfor
( CourseCode varchar(10) not null,
  StudentID varchar(10) not null,
  Qualification varchar(30) not null,
  Primary key(CourseCode, StudentID),
  Foreign key (CourseCode) References Course,
  Foreign key (StudentID) References Student );
  
```

Figure 14.12: SQL code for mapping many-to-many association to tables

You have seen the above example of many-to-many association. While mapping to the tables primary key of both the tables i.e. Course and Student are included in another new table i.e. *Registeredfor* as foreign keys along with the attributes of relationship. These keys combine to form the new table's collective primary key.

14.5.3 Mapping Ternary Associations to Tables

When the three classes are involved in a relationship, it is called ternary associations. When you are mapping these associations to the tables, all classes shall be mapped onto the distinct tables, and their relationship shall be taken by creating another table with a primary key which must be combined primary keys of all three classes. It would also include the relationship attributes such as start and end date. Let's see the following Ternary association diagram and then map these associations to tables.

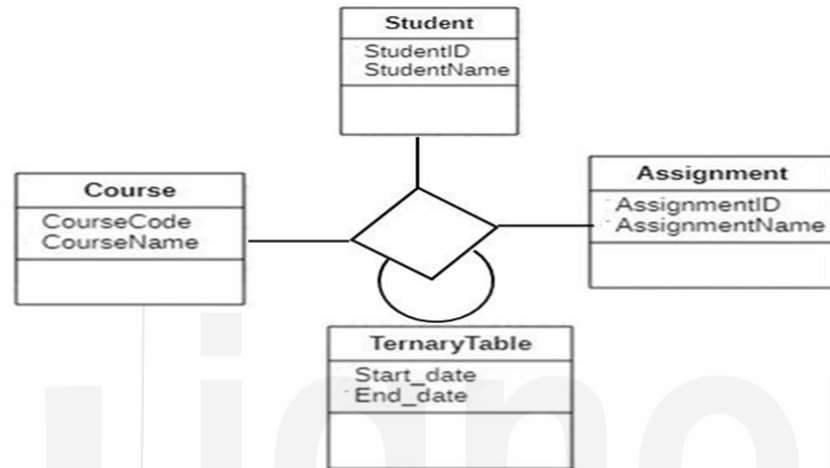


Figure 14.12: Object model for Ternary Association

The following are the table models for the above Ternary association.

Course Table Model

Attribute Name	Null	Domain
CourseCode	N	ID
CourseName	N	Name

Student Table Model

Attribute Name	Null	Domain
StudentID	N	ID
StudentName	N	Name

Assignment Table Model

Attribute Name	Null	Domain
AssignmentID	N	ID
AssignmentName	N	Name

TernaryTable Table Model

Attribute Name	Null	Domain
CourseCode	N	ID
StudentID	N	ID
AssignmentID	N	ID
Start_date	Y	Start date
End_date	Y	End date

Figure-14.13: Table models for Ternary Association.

The following is a SQL code for the above ternary association mapping to tables. After mapping, you have four RDBMS tables, as shown in figure-14.13.

```

Create table Course
( CourseCode varchar(10) not null,
  CourseName varchar(30) not null,
  Primary key (CourseCode));

Create table Student
( StudentID varchar(10) not null,
  StudentName varchar(30) not null,
  Primary key (StudentID));

Create table Assignment
( AssignmentID varchar(10) not null,
  AssignmentName varchar(30) not null,
  Primary key (AssignmentID));

Create table TernaryTable
( CourseCode varchar(10) not null,
  StudentID varchar(10) not null,
  AssignmentID varchar(10) not null,
  Start_date Datetime,
  End_date Datetime,
  Primary key (CourseCode, StudentID, AssignmentID),
  Foreign key (CourseCode) References Course,
  Foreign key (StudentID) References Student,
  Foreign key (AssignmentID) References Assignment);

```

Figure 14.15: SQL code for mapping ternary association to tables

Kindly note that the keyword VARCHAR is used in the above examples in this unit. It is a datatype in SQL databases which indicates the values of the column are variable-length character string and the length of the string are not more than the number of characters specified in parentheses.

14.6 MAPPING GENERALIZATIONS TO TABLES

As you already know, a generalization is a superclass - subclass type of relationship. This section describes different strategies which are used to map generalization relationships to tables.

The Employee superclass is a generalization of the entity subsets such as salaried or hourly employees. For example, if you are modeling an organization's database and want to store information about their employees, either salaried employees or hourly employees. In this case, you can create an Employee set that would be called as superclass and salaried/hourly employee subset is called subclasses.

The following figure-14.14 is shown above mention generalization relationship and is used to demonstrate different tactics.

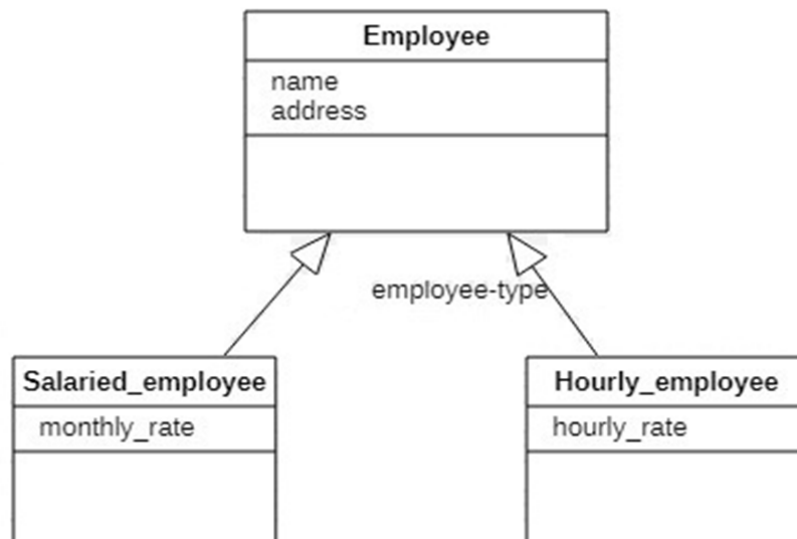


Figure 14.14: Object Model for Generalization

These are different tactics or strategies used for mapping generalizations relationship to tables.

Strategy 1 - Map the superclass and each subclass to a table

According to this strategy, all the entities in the relationship are mapped to individual tables.

To illustrate this strategy, you may map figure 14.14 as shown below in the box. You can create a table for the superclass entity as EMPLOYEE, which includes all the attributes of this employee entity in the table and underline the primary key attribute. You can also create a separate table for each subclass entity. Same as a superclass entity, include the attributes of the respective subclasses in the subclass tables. Take the primary key from the superclass table and insert into the subclass tables as a primary key and underline it.

```
Employee(empID, name, address, phoneNo)
Salaried_employee (empID, monthly_rate)
Hourly_employee(empID, hourly_rate)
```

Strategy 2 - Map each subclass to a table

Using this method, you can only map subclasses to tables. You do not map the superclass to a table. The attributes in the superclass are duplicated in all subclasses.

To illustrate this strategy, you will map figure-14.14 as shown below in the box. Here, you can create an isolated table equivalent to each subclass entity with the involvement of attributes of the corresponding subclasses in the corresponding subclass tables. You can also add the superclass entity's primary key and other attributes in all the subclass tables and underline the primary key fetched from the superclass entity to the subclass table. This strategy is most chosen when inheritance is disjoint and complete, e.g. every employee is either salaried or hourly or none.

```
Salaried_employee (empID, name, address, phoneNo, monthly_rate)
Hourly_employee(empID, name, address, phoneNo, hourly_rate)
```

In this case, only the superclass is mapped to a table and does not map the subclasses to tables. The attributes in the subclasses are brought to the superclass.

To illustrate this strategy, you may map figure 14.14 as shown below in the box. Using this strategy, you can create a single table containing the superclass entity and its attributes, along with the subclasses and their attributes. This strategy will introduce null values. When you insert a salaried employee record in the table, the *hourly_rate* column value will be null. Similarly, when you insert an hourly employee record in the table, the *monthly_rate* value will be null. This strategy may be useful when there are only two or three subclasses consisting few attributes.

```
Employee(empID, name, address, phoneNo, monthly_rate, hourly_rate)
```

14.7 INTERFACING TO DATABASES

A DBMS is a software program which is used for creating, retrieving and controlling access to permanent data. As the focus of this unit is on object mapping to databases. This section is to discuss the relationship between object orientation and relational databases and some interfacing points with the database.

Object-oriented technology is unanimously recognized methodology for developing business applications since it provides more suitable methods and services for modeling objects from the real world. Object orientation and relational databases are not compatible. Both represent two different views: in an Object-Oriented system, concepts are surrounded by the discrete objects which combine both data structure and behaviour in a single entity; in an RDBMS, all about the data or data-centric. The Object-Oriented model is suited with applications with state-specific behaviour, and the data is a secondary point. The RDBMS model is well-matched to reporting applications in which the relationships are dynamic and is still used to preserve the persistence of enterprise data.

The discussion point here is that a lot of data is stored in relational databases. If Object-Oriented applications are used to store data in RDBMS, then it is more able to read and write data in the relational database. Besides, Object-Oriented applications can share data with non-object-oriented applications by using RDBMS as the sharing mechanism.

Connecting these two standards is not an easy task and requires organized mapping from one to another. This mapping from object model to the relational database needs some rules and strategies, which we have already discussed in the above sections of this unit.

Once you have transformed object oriented data model to relational database, you shall write the interfacing code responsible for the application's functionality. This code must contain commands capable of reading and writing data from the database and interpreting it wherever requisite in terms of the model used by the program. To achieve this, you need interfacing software such as JDBC API, which supports your programming environment and allows programmers to shorten the details of individual databases and allows an application to work with a variety of databases or data sources. The JDBC API is a Java API which can be used to access any kind of tabular data, specifically data stored in a relational database. It allows Java programmers to write programs which interface with relational databases. It has three simple steps to connect a database, such as connecting to a data source, like a

Check Your Progress - 3

1. Explain the mapping of one-to-many associations to the database tables.

2. Explain how the ternary associations are mapped to the database tables.

3. Explain how the generalizations relationships are mapped to the database tables.

14.8 SUMMARY

In this unit, you have learned about the basic and relational concepts of RDBMS as well as extended 'three schema architecture' for object model, which is the base of database design. The three schema architecture provides an abstract view of data to users. This Unit has typically about the object mapping to database. You have learnt about mapping the object classes to tables. Each class transforms to one or more tables. This Unit described you the conversion process of different association to tables. These different associations are one-to-one association, one-to-many association, many-to-many association as well as ternary association. While transforming classes to tables in one-to-one association, you can use foreign key concepts. In one-to-many association, there are two options for transformation to tables such as distinct table and foreign key whereas in many-to-many association, all the classes are mapped to a distinct table.

You have also studied the mapping generalizations to tables. In this type of mapping, you have learnt different tactics or strategies. Towards this unit's end, the database interfacing has been described.

14.9 SOLUTIONS/ANSWERS TO CHECK YOUR PROGRESS

1. There are two rules of integrity in the relational model such as entity integrity and referential integrity. Entity integrity defines that each table has exactly one primary key. A primary key is a set of one or more attributes that can uniquely identify each row in a relation. Referential integrity needs a foreign key.
2. Database design is a group of methods that help the database designer design, develop, implement, and maintain the enterprise data management systems. The main objective of designing a database is to build physical as well as logical models of designs for the planned database system. Using this approach, the database designer can decide on data elements correlation and what data must be stored in the database.

There are two methods to design a database i.e. attribute driven design (ADD) and entity-driven design (EDD). In Attribute driven design, assemble a list of attributes related to the application and normalize the sets of attributes that preserve functional dependencies (FD), while in the Entity driven design method, define entities which are significant to the application and explain them.

3. The three schema architecture provides an abstract view of data to users. The database design contains three layers such as external, conceptual and internal schemas. The **external schema** is an external view which represents some portion of the entire database. It is a higher-order abstraction which the user sees. The next level is **conceptual schema** which represents data logically. Basically, it defines what data is stored really in the database. The third level of this architecture is **internal schema** which shows the lowest level of abstraction and closest to physical storage.

☞ Check Your Progress - 2

1. An object identifier (OID) is a unique name for any type of object or entity. It is a mechanism to identify an entity. When you derive a table from the class, each table has an ID for the primary key. When you derived tables from association, one or more object IDs collectively form the primary key. An ID is equal to the database concept. In a database, an object ID is a set of integers uniquely identifying each row (or record) in a table.

There are some advantages and disadvantages of using IDs. The benefit is that it does not make any changes to the data. The stability of IDs is mainly important for associations as they refer to objects. Any changes in the name of IDs, this will update many associations. So, IDs give an unvarying mechanism for referencing objects.

2. Each object class transforms into one or more tables in the database. The objects in a class may be partitioned horizontally and/or vertically. When you map class objects to relational model, you shall start with attributes of a class. You can list these attributes in the table model and insert the implicit object ID into it. While making a table model, you can add some constraints details like not null as per your need. You can also assign a primary key to each table. The Structured Query Language is used to create code for the table.
3. The object model defines the static structure of the objects in a system and their relationship. It emphasizes on logical data structures. Each object model contains classes, associations, generalizations and so on. The goal of constructing an object model is to grasp those ideas from the real world which are significant to an application. The object model graphically represents the object diagrams which contain the object. Each class defines some attributes'

Check Your Progress - 3

1. In one-to-many association, there are two options for transformation to tables: distinct table and buried foreign key.
2. When the three classes are involved in a relationship, it is called ternary associations. When you are mapping these associations to the tables, all classes are mapped onto the distinct tables. Their relationship shall be taken by creating another table with a primary key that must be combined with all three classes.
3. Whenever you map generalizations to tables, there are different tactics or strategies. As per the strategy-1, you can map the superclass and each subclass to a relational table. According to this strategy, all the entities in the relationship are mapped to individual tables. The second way to map generalizations to tables is that each subclass mapped to a table. Using this method, you can only map subclasses to tables. You do not map the superclass to a table. The attributes in the superclass are duplicated in all subclasses. You can map the superclass to a single table using the third strategy. In this case, only the superclass is mapped to a table and does not map the subclasses to tables. The attributes in the subclasses are brought to the superclass.

14.10 REFERENCES/FURTHER READING

- James Rambaugh, Michael Blaha, William Premerlam, Frederick Eddy, William Corensen, “Object Oriented Modelling and Design” PHI, New Delhi, 2004.
- Bipin C. Desai, “An Introduction to Database Systems”, Galgotia Publications, 2010.
- <http://agiledata.org/essays/mappingObjects.html>
- <https://web.csulb.edu/colleges/coe/cccs/dbdesign/dbdesign.php?page=class.php>
 - <https://docs.oracle.com/javase/tutorial/jdbc/index.html>